



RS6x 7x 毫米波雷达开发 快速入门

文档页数：42

文档版本号：1.4

发布时间：2026-6-12

版本历史

版本	版本描述	日期
1.4	将微动点云更名为存在点云； 修正了 UART-IIC 模式下 IRQ 拨码开关描述的错误； 增加 Q&A 对如何连接雷达与其它嵌入式硬件进行开发的说明； 增加执行首个工程的更多详细的步骤	2026-6-11
1.3	新增了 HOST 端对接 HIF 移植的方法	2025-12-17
1.2	新增“自适应增益”的描述； 更新了 CDK 编译工具的下载方法	2025-10-18
1.1	更新开发板“侧装竖直”的定义和配图； 修正 DUAL-UART 拨码开关描述的错误； 新增“数据输出与可视化”的介绍； 完善“软件功能移植”章节的描述	2025-10-07
1.0	初版建立	2025-09-15

目录

版本历史..... ii

目录..... iii

图片目录..... v

表格目录..... vi

1 目的和应用范围..... 1

2 概述..... 2

 2.1 开发套件..... 2

 2.1.1 硬件开发套件..... 2

 2.1.2 软件开发套件..... 3

 2.2 相关资料..... 4

3 软件环境搭建..... 5

 3.1 编译工具链准备..... 5

 3.2 工具及驱动安装..... 5

4 在开发板上执行首个工程..... 6

 4.1 硬件环境搭建..... 6

 4.2 编译..... 7

 4.2.1 目录结构..... 8

 4.2.2 选择板级宏配置..... 8

 4.2.3 编译..... 9

 4.3 烧录..... 10

 4.4 运行..... 11

5 在目标硬件执行首个工程..... 15

 5.1 基于 Sample 建立工程..... 15

 5.2 建立目标硬件的开发模板..... 15

 5.3 通信功能移植..... 18

 5.3.1 通信接口功能配置..... 18

 5.3.2 典型 HIF 应用场景..... 20

 5.3.3 通信硬件 IO 配置..... 20

 5.4 雷达功能移植..... 21

 5.5 调试接口..... 21

 5.5.1 TRACE_IO..... 21

 5.5.2 日志系统..... 22

 5.5.3 PSIC-调试协议..... 23

 5.6 可靠性设计..... 25

5.6.1 看门狗..... 25

5.6.2 Task Monitor.....26

5.6.3 心跳包..... 26

5.6.4 Response Count..... 26

5.7 嵌入式资源配置与验证..... 27

5.7.1 代码资源配置方式..... 27

5.7.2 嵌入式资源验证..... 28

5.7.3 计算时间测量..... 30

5.8 低功耗功能移植与验证..... 31

5.8.1 低功耗模式配置..... 31

5.8.2 功耗验证..... 31

5.9 SDK 其他修改..... 32

6 雷达调试方法..... 33

6.1 雷达配置接口的生成..... 33

6.2 测试与验证..... 33

7 Q&A..... 34

图片目录

图 1-1 推荐的雷达评估、设计和开发流程和工具.....	1
图 2-1 毫米波雷达模组与开发板展示.....	3
图 2-2 cdk 工具下载.....	4
图 3-1 setup.exe.....	5
图 3-2 开发套件 UART 设备信息.....	5
图 4-1 硬件开关示意图.....	6
图 4-2 通过工程配置切换宏配置.....	9
图 4-3 IDE 界面.....	10
图 4-4 Rebuild 界面.....	10
图 4-5 连接 RadarDebugTool 工具.....	11
图 4-6 开发板“侧装竖直”安装.....	12
图 4-7 上位机启动雷达.....	13
图 4-8 上位机显示点云.....	13
图 4-9 上位机显示调试信息.....	14
图 4-10 上位机停止雷达数据上报.....	14
图 5-1 添加 myboard 文件夹.....	16
图 5-2 boards 文件夹结构.....	16
图 5-3 board_config 宏定义.....	17
图 5-4 配置 MYBOARD.....	17
图 5-5 在 myboard_config.h 中修改相关的宏.....	18
图 5-6 配置自定义模板全局定义宏.....	18
图 5-7 UART PinMux 修改.....	21
图 5-8 PSIC-调试协议.....	23
图 5-9 frame_info 工具显示.....	25
图 5-10 链接文档指定路径.....	28
图 5-11 map 文件内存分布.....	28
图 5-12 CONFIG_DEBUG_SIZE_LEVEL=1 资源统计结果示例.....	29
图 5-13 CONFIG_DEBUG_SIZE_LEVEL=2 资源统计结果示例.....	29
图 5-14 低功耗和 CPU 性能配置.....	31
图 7-1 Clean Project.....	34
图 7-2 连接模组引脚与开发板硬件的电阻.....	35

表格目录

表 4-1	不同模组通信模式功能说明.....	7
表 4-2	不同模式拨码开关设置.....	7
表 4-3	Tracking SOC 配置.....	9
表 4-4	Tracking 默认波形参数.....	11
表 5-1	shell/LOG/HIF 支持映射的硬件.....	19
表 5-2	推荐的通信功能配置.....	19
表 5-3	典型场景下不同接口传输能力.....	20
表 5-4	默认的 CPU 频率.....	32

1 目的和应用范围

本文档旨在详细描述使用正和微芯 RS6x_7x 系列毫米波雷达开发工具包的方法，通过对一个典型示例的讲解和移植的描述，让开发者能基于示例在目标硬件上快速完成软硬件环境搭建和首个工程的跑通。

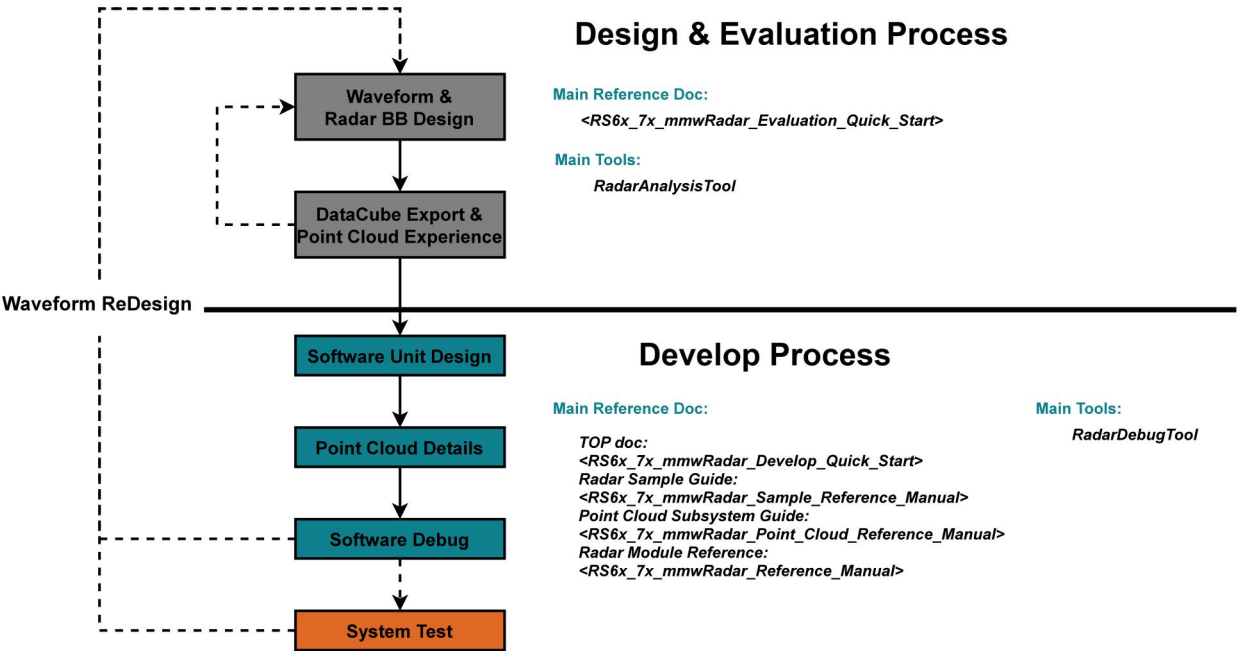


图 1-1 推荐的雷达评估、设计和开发流程和工具

推荐开发者按照如图 1-1 所示的方式使用 SDK。SDK 将开发流程划分为设计评估阶段和开发阶段，并提供相应的工具。

在评估阶段，主要使用 RadarAnalysisTool，并搭配 RadarAnalysisTool 固件进行性能评估，通过调节工具提供的参数配置定义雷达波形、通过工具提供的数据采集功能将波形或点云导入上位机，开发者可在上位机中验证并调优自选算法，直至雷达波形和自选算法协同达到预期效果后，可进入开发阶段。

在开发阶段，开发者借助 SDK 提供的 API 和 RadarDebugTool 等工具，并参考 Sample 示例与 SDK 使用说明文档，将评估阶段选定的雷达波型和自选算法部署进嵌入式系统，并进行测试、验证和系统优化。

本文档支持的芯片包括：RS6x 和 RS7x 系列所有芯片。

2 概述

本文档主要面向首次接触 RS6x_7x 系列雷达传感器芯片的开发者，展示如何使用 RS6x_7x 系列开发包在目标硬件上快速完成“Tracking”工程的搭建。“Tracking”包含了基于 HIF（Host InterFace）应用协议的数据通信链路配置、点云计算和点云数据上报，可作为开发者的首个入门工程。

本文档首先指导开发者如何在开发板上完成软件硬件环境搭建，跑通“Tracking”工程，并配合 RadarDebugTool 工具进行数据可视化。

随后，本文档将指导开发者如何通过修改关键板级配置，快速将“Tracking”部署至目标硬件上。

最后，本文将介绍 SDK 提供的调试手段和工具，这些工具可协助开发者进行数据可视化、参数调优、单元测试和算法优化。

若开发者需要了解如何使用和配置 Sample，请参考《RS6x_7x_毫米波雷达_Sample_参考手册》；

若开发者需要了解如何使用 SDK 提供的点云计算参考流程，请参考《RS6x_7x_点云流程_参考手册》；

若开发者需要了解如何使用雷达驱动及其特性，请参考《RS6x_7x_毫米波雷达_参考手册》；

若开发者需要了解如何配置和使用 HIF 通信协议，请参考《RS6x_7x_HIF 开发_快速入门》。

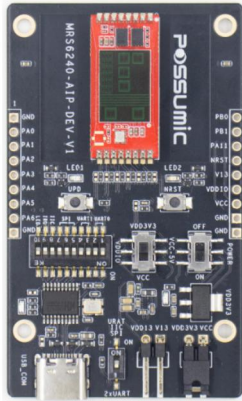
2.1 开发套件

开发套件包括“硬件开发套件”和“软件开发套件”两部分。

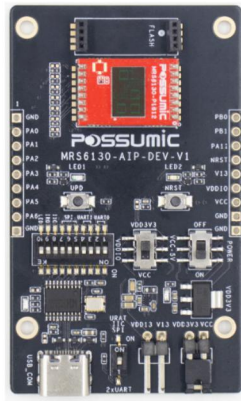
2.1.1 硬件开发套件

1. MRS6130_P1812/MRS6130_P1806/MRS6240_P2512/MRS6241_P2828_M62/MRS6241_P2840_M81：毫米波雷达模组（红色）；
2. MRS6130-AIP-DEV-V1/MRS6240-AIP-DEV-V1/MRSx241-DEV-V1：承载模组的开发板（黑色）；
3. USB Type-C 数据线：用于连接 PC 和开发板的连接线。

CHIP: RS6240
MODULE: MRS6240-P2512
DEV BOARD: MRS6240-AIP-DEV-V1



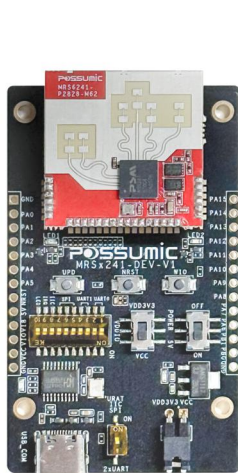
CHIP: RS6130
MODULE: MRS6130-P1812
DEV BOARD: MRS6130-AIP-DEV-V1



CHIP: RS6130
MODULE: MRS6130-P1806
DEV BOARD: MRS6130-AIP-DEV-V1



CHIP: RS6241
MODULE: MRS6241-P2828-M62
DEV BOARD: MRSx241-DEV-V1



CHIP: RS6241
MODULE: MRS6241-P2840-M81
DEV BOARD: MRSx241-DEV-V1

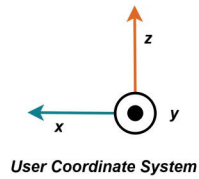
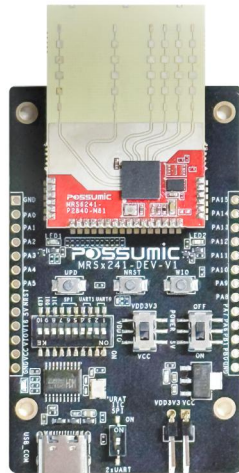


图 2-1 毫米波雷达模组与开发板展示

2.1.2 软件开发套件

软件开发套件目录结构如下：

<RS6x_7x 系列开发包>/

- ├── Software_Kit /* 主要面向软件工程师开发使用 */
 - ├── 01_Document /* 各种软件的指导文档 */
 - ├── 02_SDK /* 软件源代码包，编译工具链 */
 - ├── 03_Tool /* USB 驱动，烧录工具，雷达 GUI 工具等 */
 - └── 04_Image /* 预编译的 Demo 固件 */
- └── Hardware_Kit /* 主要面向硬件工程师开发、评估使用 */
 - └── 01_Document /* 各种硬件的指导文档 */

2.2 相关资料

- 编译环境工具：访问正和微芯官网 <https://www.possumic.com/rs6130.html>



图 2-2 cdk 工具下载

如图 2-2 所示，“技术资料”中点击“产品包”，注册用户后，点击下载 cdk-windows.zip。

- 示例工程：
 - ◆ mmwave 工程：/Software Kit/02 SDK/psdf_sdk/project/mmwave/mmwave
 - ◆ mmwave 应用工程：/Software Kit/02 SDK/psdf_sdk/project/mmwave/mmwave_application
- RadarDebugTool：</Software Kit/03 Tool/RadarDebugTool>
- 串口驱动：</Software Kit/03 Tool/USB Serial Driver>
- PC 串口工具：请开发者自备。

3 软件环境搭建

在开始使用 SDK 编译代码之前，需要准备编译环境和相关软件。

3.1 编译工具链准备

- 1. 解压 cdk，进入目录并双击 “setup.exe” ；

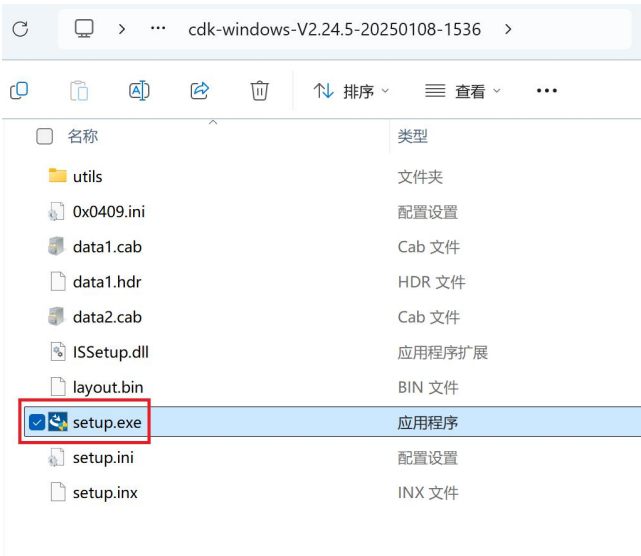


图 3-1 setup.exe

- 2. 进行安装。

3.2 工具及驱动安装

开发套件的底板已集成 USB 转串口硬件，需要安装驱动即可使用串口。

驱动路径： [/Software Kit/03 Tool/USB Serial Driver](#)

安装成功后，接上开发套件至 PC，将看到如下 1 个串口（UART-SPI/UART-I2C 模式）或 2 个串口（DUAL-UART）。



图 3-2 开发套件 UART 设备信息

4 在开发板上执行首个工程

本章以“Tracking”工程为例，介绍代码结构、如何编译工程源码、烧录固件至开发板并在 RadarDebugTool 上进行配置和可视化。“Tracking”工程提供了基于 HIF 的配置链路、点云数字信号处理流程和使用 RadarDebugTool 工具进行点云数据可视化等 3 大重要功能的展示。

“Tracking”工程位于：/Software Kit/02_SDK/psdf_sdk/project/mmwave/mmwave/Tracking

4.1 硬件环境搭建

所有开发板上均具有两组功能相同的拨码开关，分别以橙色框和绿色框表示。橙色框中的一组拨码开关为 FUNC SEL，绿色框中拨码开关为 MODE SEL，在开发过程中常用的还有标注为 POWER 的单刀双掷开关，和标注为 UPD(UPDATE)的轻触开关。

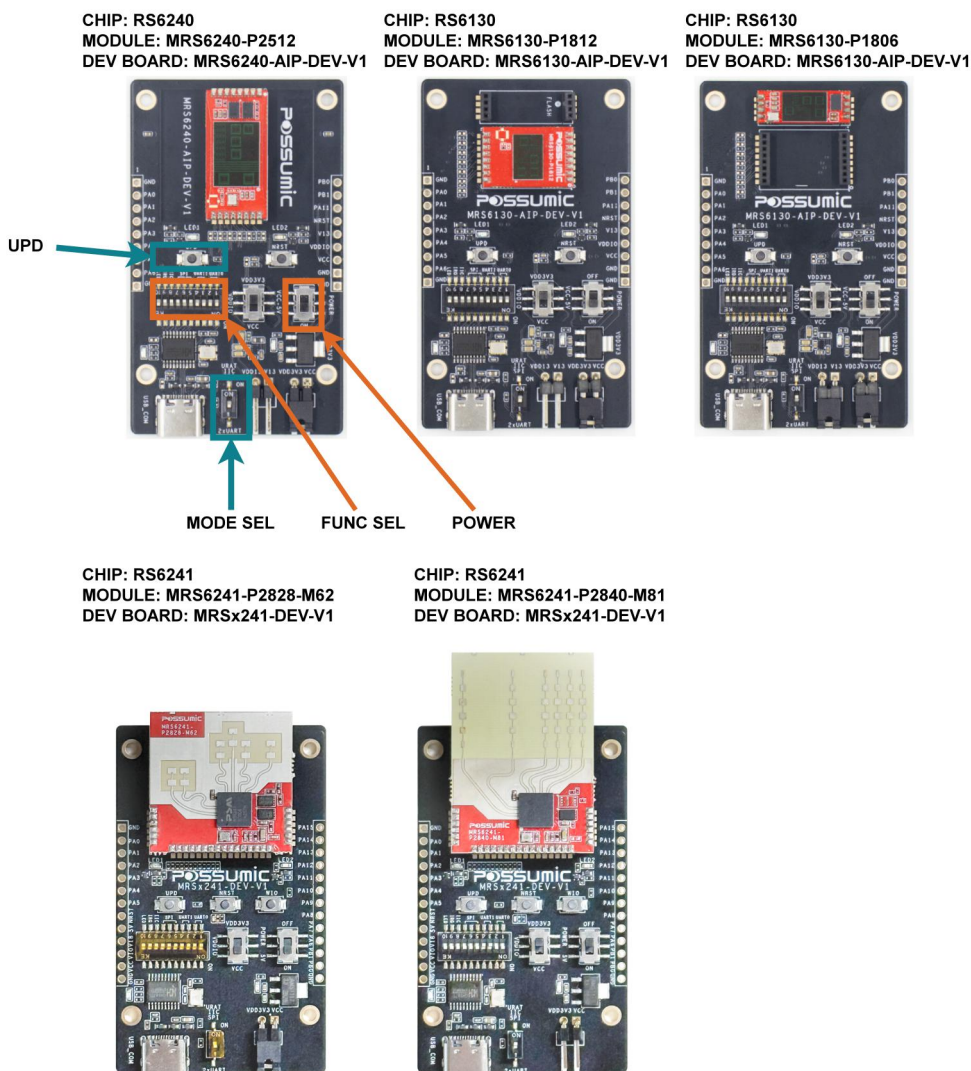


图 4-1 硬件开关示意图

MRS6240-AIP-DEV-V1、MRS6130-AIP-DEV-V1 和 MRSx241-DEV-V1 开发板支持的三种通信模式，分别称为“DUAL-UART”、“UART-SPI”和“UART-I2C”模式，由 FUNC SEL 和 MODE SEL 两组拨码开关控制。

表 4-1 不同模组通信模式功能说明

模式选择	DUAL-UART			UART-SPI/UART-IIC		
硬件接口	UART0	UART1	SPI	UART0	UART1	SPI/IIC
MRS6130-P1806	Tx/Rx	Tx ONLY	-	Tx/Rx	-	NOT SUPPORT
MRS6130-P1812	Tx/Rx	Tx/Rx	-	Tx/Rx	-	Tx/Rx
MRS6240-P2512	Tx/Rx	Tx/Rx	-	Tx/Rx	-	Tx/Rx
MRS6241-P2828	Tx/Rx	Tx/Rx	-	Tx/Rx	-	Tx/Rx
MRS6241-P2840	Tx/Rx	Tx/Rx	-	Tx/Rx	-	Tx/Rx

不同模式对应的拨码开关位置如下表所示：

表 4-2 不同模式拨码开关设置

开关位	FUNC SEL										MODE SEL
	1	2	3	4	5	6	7	8	9	10	1
	UART0	UART0	UART1	UART1	SPI	SPI	SPI	IIC	IRQ	LED	2xUART
DUAL-UART	ON	ON	ON	ON	OFF	OFF	OFF	OFF	OFF	ANY	OFF
UART-SPI	ON	ON	OFF	OFF	ON	ON	ON	OFF	ON	ANY	ON
UART-IIC	ON	ON	OFF	OFF	OFF	OFF	OFF	ON	ON	ANY	ON

上表中，“ON”表示拨码开关拨向拨码开关上标识“ON”的位置。

SPI 最高支持 60Mbps（受 CPU 频率影响）的传输速率，大约支持以 10Hz 帧率传输 256K DataCube。UART 最高支持 2Mbps。（由于部分电脑板卡限制，2Mbps 的波特率可能不稳定，若发现 2Mbps 波特率出现误码，建议使用 1Mbps 的波特率）。

“Tracking”参考工程默认配置：**MRS6130-P1806 模组配置为“DUAL-UART”，UART0 波特率为 115200，用于固件下载和打印，UART1 波特率为 1Mbps，用于数据上报，MRS6130-P1812、MRS6240-P2512、MRS6241-P2828 和 MRS241-P2840 模组配置为“UART-SPI”模式，UART 波特率为 115200。**

本文以 MRS6240-P2512 为示例，若开发者使用 MRS6130-P1806 模组开发，后续涉及 SPI 部分的操作，均对应为“DUAL-UART”。

注意，如果切换了“MODE SEL”拨码开关，需要对模组重新上下电（拨动 POWER 拨码开关），模式切换才会生效。

若需要了解更多关于开发板的硬件信息，请参考对应模组的开发板使用手册。

4.2 编译

在执行后续步骤前，首先确保已按照第 3 章的步骤完成软件环境搭建。

4.2.1 目录结构

SDK 源码目录结构如下，找到 Tracking 工程目录。

```
<home>/Software_Kit/02_SDK/
psdf_sdk                      /* 软件代码 */
|__ kernel                    /* RTOS 源码和 OS 抽象层 */
|__ libs                      /* 存放 静态库(.a)，二进制文件(.img) */
|__ platform                  /* 芯片平台相关驱动和配置文件 */
|__ project                   /* 应用工程 */
|   |__ mmwave                /* radar 相关工程 */
|   |   |__ common            /* 示例工程通用文件 */
|   |   |__ radar_analysis    /* 雷达分析工程，配合 RadarAnalysis 工具使用 */
|   |   |__ mmwave            /* Data Cube 上报工程 */
|   |       |__ ReportDataCube1D /* 1D 帧模式 Data Cube 上报工程 */
|   |       |__ ReportDataCube2D /* 2D 帧模式 Data Cube 上报工程 */
|   |       |__ mmwave_application
|   |       |__ Posture        /* 姿态点云工程示例 */
|   |       |__ Sleep          /* 睡眠检测工程示例 */
|   |       |__ Tracking       /* 跟踪轨迹工程示例 */
|   |__ subsys                /* 子系统工程 */
|__ subsys                    /* 功能较独立的子系统 */
|   |__ hif                   /* HostInterface 接口子系统 */
|   |__ mmw                   /* 雷达子系统 */
|       |__ mmw_algorithm      /* 雷达基础算法 */
|       |__ mmw_application    /* 雷达应用算法，包含移动点云和存在点云 */
|__ tools                     /* 工具脚本 */
```

4.2.2 选择板级宏配置

Tracking 通过在工程配置中的预定义宏选择开发板，不同的开发板会初始化不同的 IO 和外设、CPU 系列和配置不同的 CPU 频率。目前 RS61xx 系列仅可使用 CPUS，RS62xx 系列仅可使用 CPUF（支持 DSP 加速、更高的主频）。

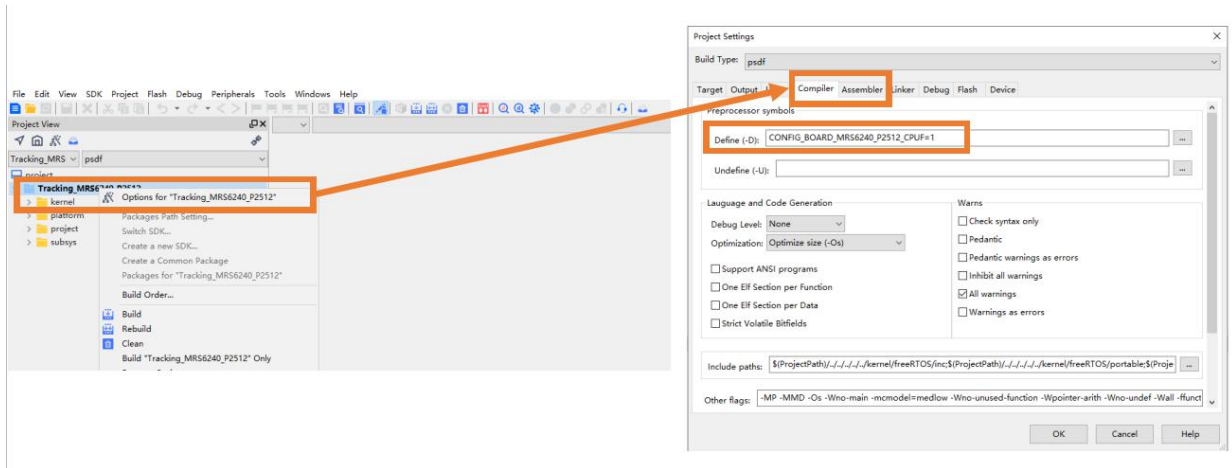


图 4-2 通过工程配置切换宏配置

注意请在“Compiler”和“Assembler”两个选项卡上均选择正确的板级宏定义。

cdk_6240_cpuf 工程默认选择的板级宏定义为 CONFIG_BOARD_MRS6240_P2512_CPUF=1。

cdk_6130_1812_cpus 工程默认选择的板级宏定义为 CONFIG_BOARD_MRS6130_P1812=1。

cdk_6130_1806_cpus 工程默认选择的板级宏定义为 CONFIG_BOARD_MRS6130_P1806=1。

cdk_6241_2828_xip_cpuf 工程默认选择的板级宏定义为 CONFIG_BOARD_MRS6241_P2828_M62_CPUF=1。

cdk_6241_2840_xip_cpuf 工程默认选择的板级宏定义为 CONFIG_BOARD_MRS6241_P2840_M81_CPUF=1。

目前 SDK 中预置了 5 种支持的板级配置：

- CONFIG_BOARD_MRS6130_P1812
- CONFIG_BOARD_MRS6130_P1806
- CONFIG_BOARD_MRS6240_P2512_CPUF
- CONFIG_BOARD_MRS6241_P2828_M62_CPUF
- CONFIG_BOARD_MRS6241_P2840_M81_CPUF

可选的板级宏定义请参考“[/Software Kit/02 SDK/psdf sdk/platform/boards/](#)”中的配置头文件，它们被总配置文件：“common.h”所包含，“common.h”位于“[/Software Kit/02 SDK/psdf sdk/platform/utis/](#)”。

表 4-3 Tracking SOC 配置

BOARD	CPU 频率	通信模式	打印输出接口	数据输出接口	移动点云输出
MRS6130-P1806	128MHz	DUAL-UART	UART1@115200	UART0 @1Mbps	支持
MRS6130-P1812	128MHz	UART-SPI	UART0@115200	SPI	支持
MRS6240-P2512	256MHz	UART-SPI	UART0@115200	SPI	支持
MRS6241-P2828	256MHz	UART-SPI	UART0@115200	SPI	支持
MRS6241-P2840	256MHz	UART-SPI	UART0@115200	SPI	支持

4.2.3 编译

1. 进入目录“[/Software Kit/02 SDK/psdf sdk/project/mmwave/mmwave_application/Tracking](#)”，根据使用芯片型号进入相应工程目录；

2. 双击“project.cdkproj”打开工程，如图 4-3 所示；

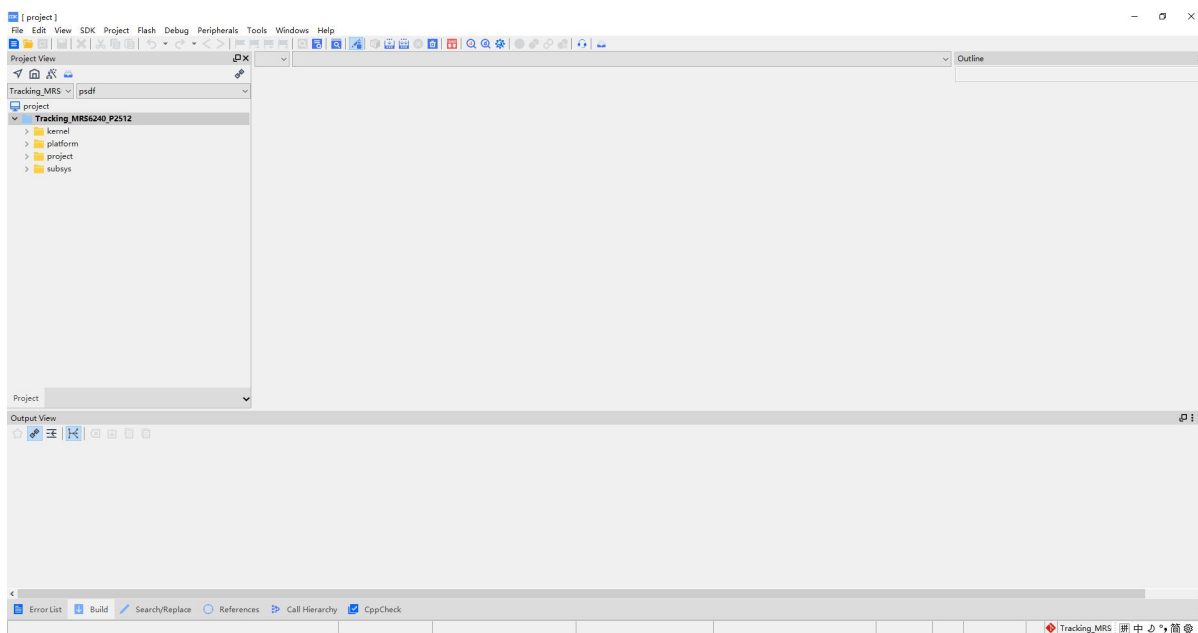


图 4-3 IDE 界面

3. 单击 IDE 工具栏的“Rebuild Project”按钮，重新编译工程，如图 4-4 所示，需注意编译和 img 打包输出信息，编译完成的数据位于工程文件(.cdkproj)目录下的 output 文件夹中。

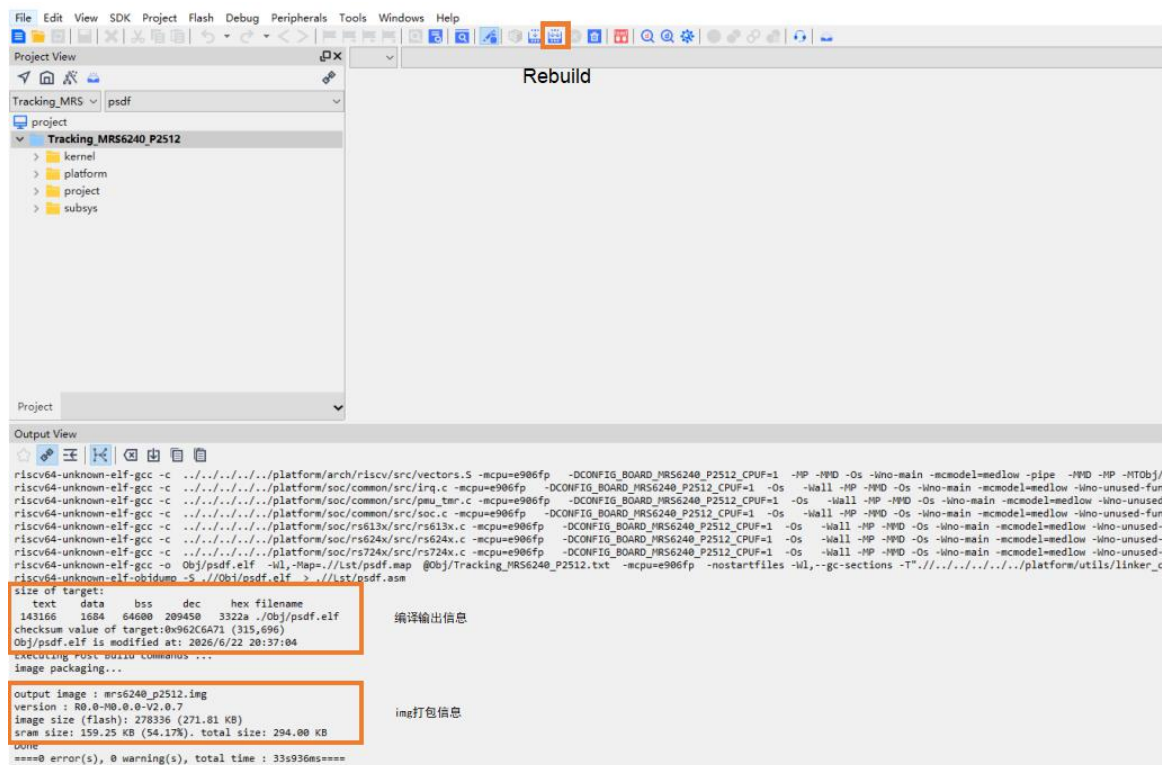


图 4-4 Rebuild 界面

注：工程配置及 IDE 详细指南，请参考《RS6x_7x_SDK_参考手册》。

4.3 烧录

烧录是通过 RadarDebugTool 工具，将编译输出的 img 加载到开发套件的 spi-Flash 中的过程。

1. 开发套件进入“烧录模式”；
 - (1) 根据表 4-2 的配置，将开关配置为“UART-SPI”模式（MRS6130-P1806 模组配置为“DUAL-UART”模式）；
 - (2) 将“POWER”开关拨向“OFF”（power LED 熄灭）；
 - (3) 按下“UPD”按键并保持，将“POWER”开关拨向“ON”（power LED 点亮）；
 - (4) 保持“UPD”按键按下约 2 秒后松开；
2. 烧录工具连接烧录：
 - (1) 若使用“UART-SPI”模式，选择“通信方式”为“SPI”，选择 SPI 端口，SPI 频率选择“60MHz”；
 - (2) 若使用 MRS6130-P1806，选择通信方式为“UART”一般为 COM 号较大的口，波特率选择 115200；
 - (3) 点击“Connect”连接雷达，然后点击“Download”选择需要下载的固件，则自动开始烧录，当进度条到 100%，则烧录成功；
 - (4) 烧录完成后，需要按“NRST”为雷达复位，使雷达使用新的固件开始工作。

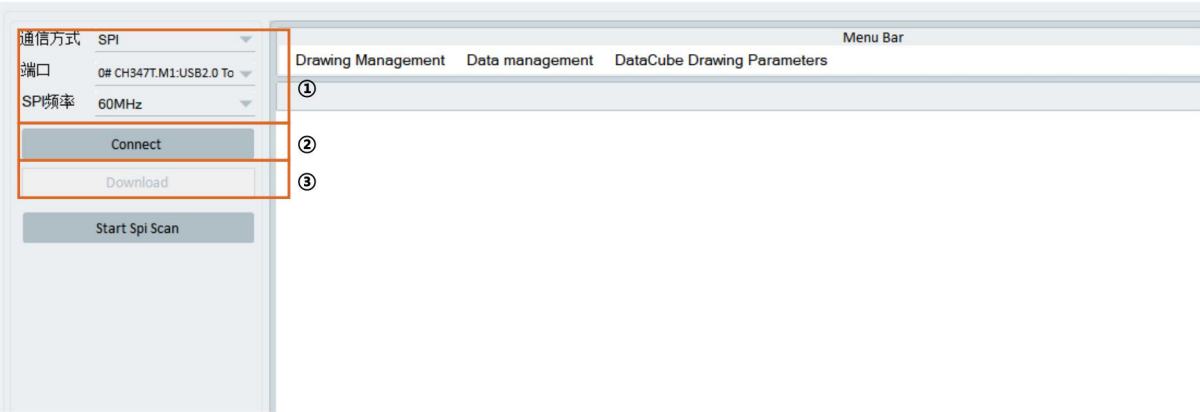


图 4-5 连接 RadarDebugTool 工具

若连接失败，可能串口号选择错误或开发套件未进入“烧录模式”，请重复步骤 1（开发套件进入“烧录模式”）和通过 PC 设备管理器确认串口号或更换串口号。

若烧录过程失败，可能为串口波特率过高，请降低波特率再试。

4.4 运行

“Tracking”示例主要展示 HIF 通信链路和点云算法。针对不同的模组类型，由于接口传输速率限制，Tracking 工程配置了不同的默认波形参数。

表 4-4 Tracking 默认波形参数

BOARD	距离分辨率	最大探测距离	速度分辨率	最大无模糊速度	帧周期
MRS6130-P1806	0.08m	20.48m	0.2m/s	3.2m/s	100ms
MRS6130-P1812	0.08m	20.48m	0.2m/s	3.2m/s	50ms
MRS6240-P2512	0.08m	20.48m	0.2m/s	3.2m/s	50ms
MRS6241-P2828	0.08m	20.48m	0.2m/s	3.2m/s	50ms

MRS6241-P2840	0.08m	20.48m	0.2m/s	3.2m/s	50ms
---------------	-------	--------	--------	--------	------

根据使用的芯片类型，根据表 4-3 的“通信模式”栏确认通信模式，并根据表 4-2 确认拨码开关已经拨在正确的位置。

“Tracking”支持雷达“侧装竖直”安装，不同开发板“侧装竖直”的摆放如图 4-6 所示：

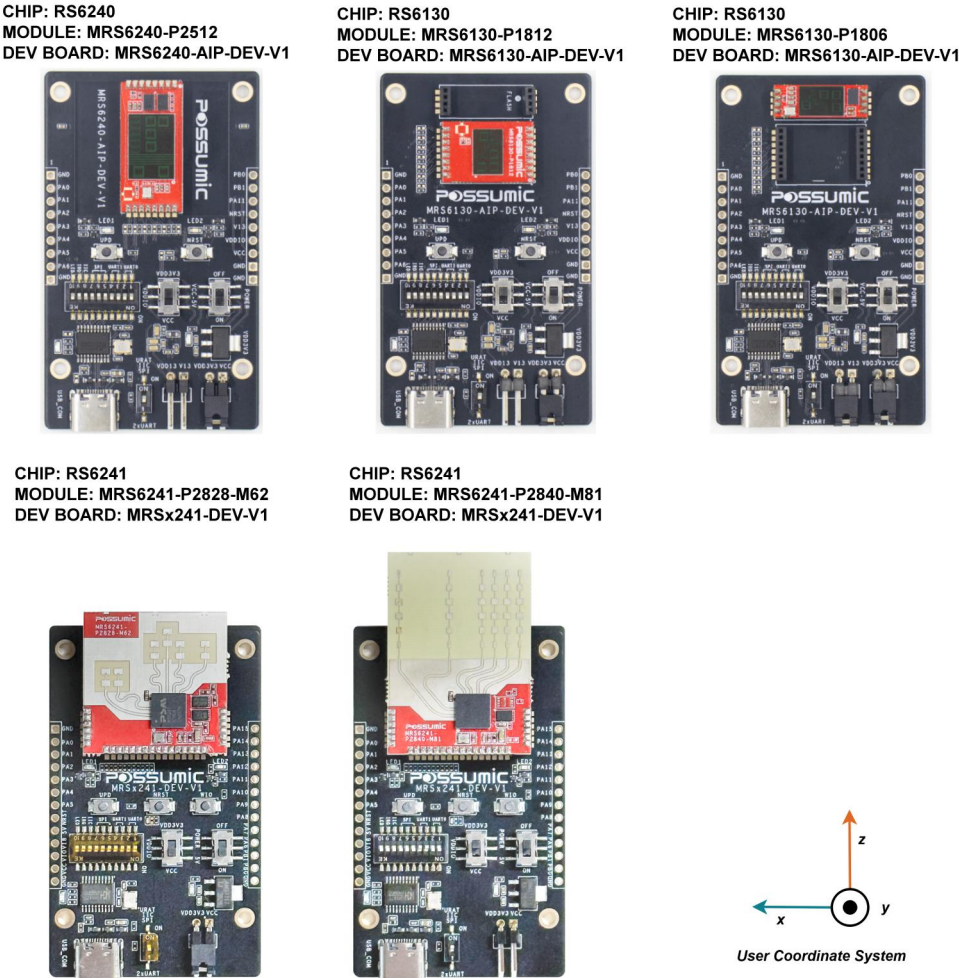


图 4-6 开发板“侧装竖直”安装

操作步骤如下：

1. 上位机通过按钮启动雷达数据上报，若使用 MRS6130-P1806，通信方式选择“UART”，并选择 UART0 对应的 COM 口（一般对应 COM 号较大的 COM 口），波特率为 1Mbps：



图 4-7 上位机启动雷达

- 选择连接到 PC 的端口，点击 Connect 按钮连接雷达；
- 点击 Download 按钮，烧录固件。(若雷达中已经烧录过固件，可以跳过此步骤)；
- 点击 Start Scan 按钮，启动上位机数据接收。
- 若使用 MRS6130-P1806，需要在完成固件烧录后，将端口切换至 UART0 对应的串口以接收数据。

2. 在 RadarDebugTool 上选择显示点云、DataCube 信息或调试信息。在绘图窗口点击右键，并选择希望展示的三视图数据，并选择绘图方式为'Scatter Plot'：

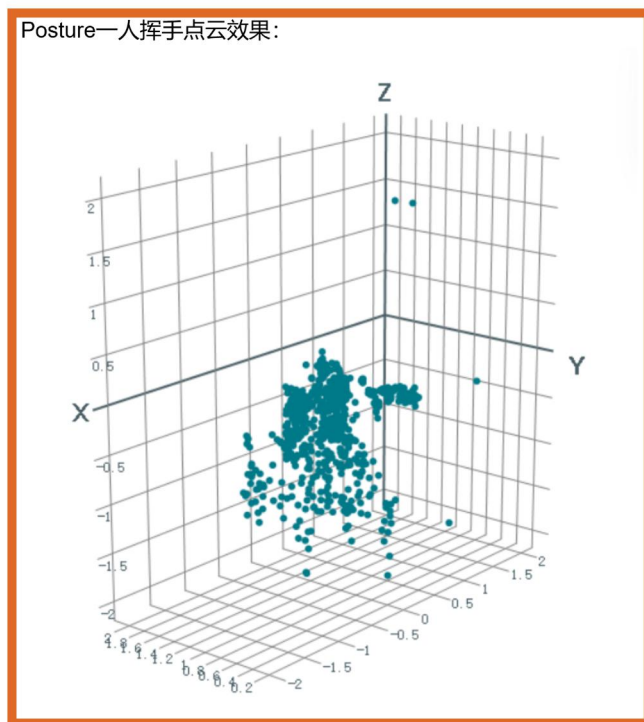
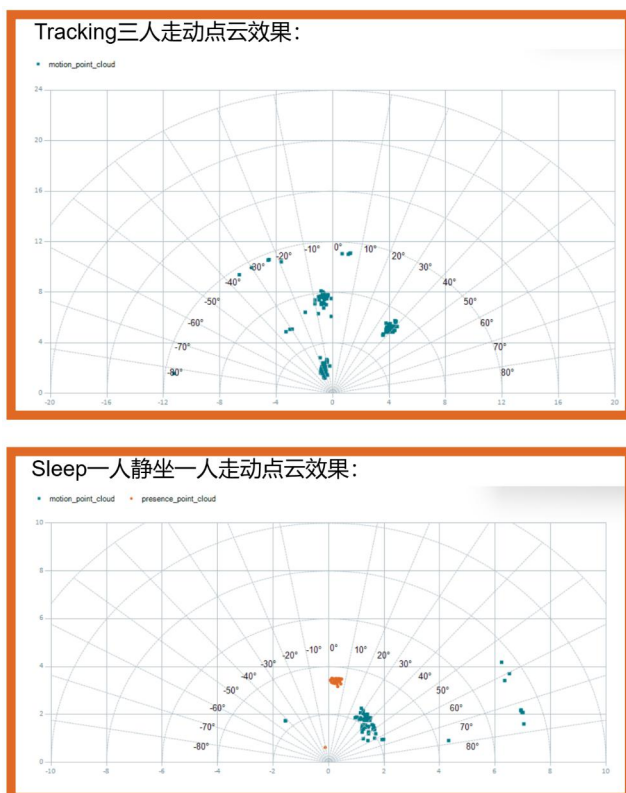


图 4-8 上位机显示点云

在图 4-8 中分别展示了三个 Sample 在典型应用场景下的点云效果，左侧两个为点云的二维“X-Y”视图，右侧为点云的三维视图。更多的 RadarDebugTool 工具的使用请参考《RS6x_7x_RadarDebugTool_使用手册》。

3. 开发者可在任意时刻切换至 Setting 界面，选择需要的波形后，点击“一键配置”配置雷达波形，观察点云情况。需要注意的是，“雷达天线工作模式”选择需要区分 RS6130 和 RS6240，目前对于 RS6130 仅支持配置为 **1T3R**，对 RS6240 仅支持配置为 **2T4R**。在图 4-9 中的红色方框内，可显示上报的数据。



图 4-9 上位机显示调试信息

4. 上位机通过点击“Stop Scan”按钮停止接收雷达数据。



图 4-10 上位机停止雷达数据上报

5 在目标硬件执行首个工程

本章主要说明如何将“Tracking”工程移植到目标硬件，主要面临的挑战在于数据传输接口的修改和适配。

HIF 协议（Host Interface）是 SDK 提供的标准数据结构，它位于通信模型中的应用层，SDK 提供的工具都支持 HIF 协议，并为开发者提供了 HOST 端的示例代码，可方便地将 HIF 协议在 HOST 主机上实现，强烈建议开发者直接使用 HIF 协议进行数据传输。若需要了解更多 HIF 的细节，请参考《RS6x_7x_HIF 开发_快速入门》。

点云数字信号处理与硬件平台无关，因此不需要考虑点云数字信号处理算法的移植。若需要了解点云数字信号处理算法的具体实现，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》。

目标硬件的设计需要遵守一定的约束，相关约束请参考《RS6xxx_硬件设计和 PCB Layout_指导手册》。

5.1 基于 Sample 建立工程

开发者首先需要将“Tracking”工程拷贝并重命名至新的目录。

需要特别注意的是，通过 SDK 接口或开源代码提供的数字信号处理算法可能随 SDK 的更新而进行细微调整，这会导致点云输出特性发生改变。若开发者需要对 SDK 提供的数字信号处理算法进行定制，则需要将有改动的部分独立出来以实现产品算法和 SDK 的解耦。

开发者在每次 SDK 更新时需要确认 Release Note，检查 SDK 是否修复了原有接口的 BUG，并评估点云输出特性的变化对算法的影响。

关于 RS6130、RS6240 以及 RS6241 系列芯片更多工程创建的方法，请参考《RS6x_7x_SDK_参考手册》。

5.2 建立目标硬件的开发模板

SDK 采用“开发模板”进行芯片的 PINMUX 配置和功能模块配置。开发者需要根据目标硬件，建立自己的开发模板。默认的开发模板位于“platform/boards”中，假设目标板名称为“myboard”，建立开发模板方法如下。

1. 进入“platform/boards”目录，复制“mrs6240_p2512”（用于 RS6240 系列）、“mrs6130_p1812”（用于 RS6130 系列）、“mrs6241_p2828”（用于 RS6241_P2828）或者“mrs6241_p2840”（用于 RS6241_P2840）文件夹并重命名为“myboard”。进入“myboard”目录修改“mrs6240_p2512.c”和“mrs6240_p2512_config.h”为“myboard.c”和“myboard_config.h”。并将 myboard 文件夹通过“Add Source Folder”加入工程：

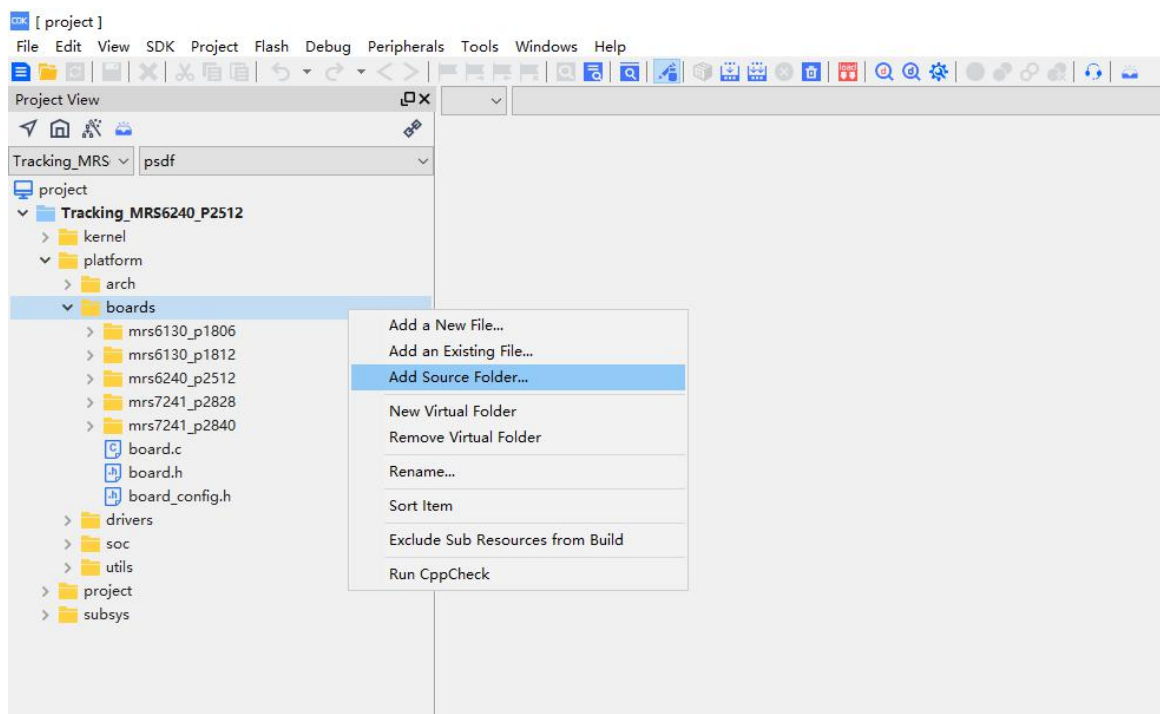


图 5-1 添加 myboard 文件夹

2. 此时，工程的“boards”目录如图所示：

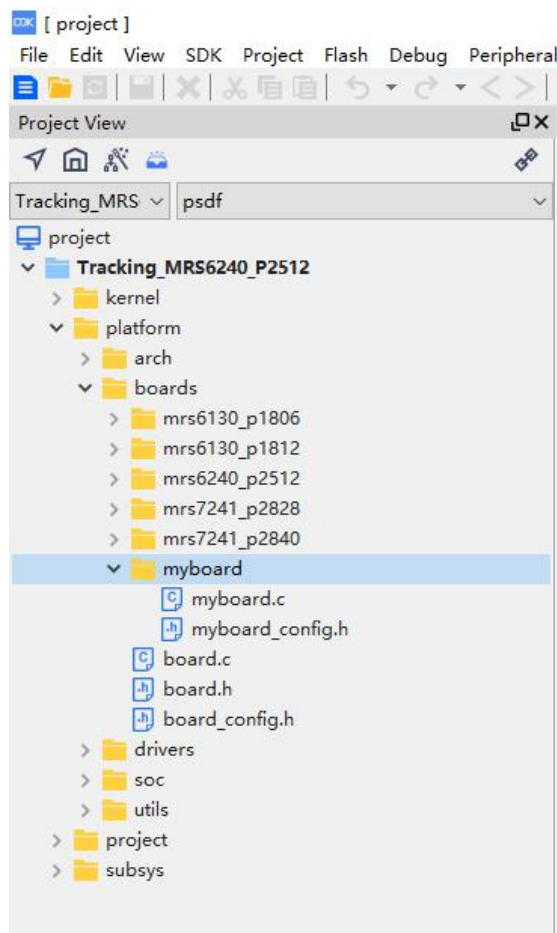


图 5-2 boards 文件夹结构

3. 打开“board_config.h”，增加宏和头文件包含：

```

#ifndef CONFIG_BOARD_MRS6242_P2512_CPUS
#define CONFIG_BOARD_MRS6242_P2512_CPUS ..... 0
#endif

#ifndef CONFIG_BOARD_MRS6242_P2512_CPUF
#define CONFIG_BOARD_MRS6242_P2512_CPUF ..... 0
#endif

#ifndef CONFIG_BOARD_MYBOARD
#define CONFIG_BOARD_MYBOARD ..... 0
#endif

/* Private includes.
 *
 */

#ifdef CONFIG_BOARD_MRS6130_P1806
#include "mrs6130_p1806/mrs6130_p1806_config.h"
#elif CONFIG_BOARD_MRS6130_P1812
#include "mrs6130_p1812/mrs6130_p1812_config.h"
#elif CONFIG_BOARD_MYBOARD
#include "myboard/myboard_config.h"
#elif ((CONFIG_BOARD_MRS6240_P2512_CPUS) || (CONFIG_BOARD_MRS6240_P2512_CPUF))
#include "mrs6240_p2512/mrs6240_p2512_config.h"
#elif ((CONFIG_BOARD_MRS6242_P2512_CPUS) || (CONFIG_BOARD_MRS6242_P2512_CPUF))
#include "mrs6242_p2512/mrs6242_p2512_config.h"
#else
#error "No board has been selected!!!"
#endif

```

图 5-3 board_config 宏定义

4. 打开 “myboard.c”，修改开关宏和 log 配置宏：

```

#include "soc_config.h"

#ifdef CONFIG_BOARD_MYBOARD

#include "hal_types.h"
#include "hal_dev.h"
#include "hal_board.h"

#include "hal_power.h"
#include "hal_clock.h"
#include "hal_gpio.h"
#include "hal_uart.h"
#include "hal_spi.h"
#include "hal_i2c.h"
#include "hal_flash.h"
#include "ll_rcc_dev.h"
#include "ll_utils.h"
#include "log_port.h"

#ifdef CONFIG_SHELL
#include "shell.h"
#endif

#ifndef CONFIG_BOARD_LOG_LEVEL
#define CONFIG_BOARD_LOG_LEVEL ..... LEVEL_ERR
#endif

#define LOG_MODULE ..... "myboard"
#define LOG_LEVEL ..... CONFIG_BOARD_LOG_LEVEL
#include "log.h"

```

图 5-4 配置 MYBOARD

5. 打开 “myboard_config.h”，修改自定义模板相关的宏：

```
#ifndef _MYBOARD_CONFIG_H
#define _MYBOARD_CONFIG_H

/* Includes.
 *
 */

/* Exported macro.
 *
 */

if CONFIG_BOARD_MYBOARD
#define CONFIG_BOARD_STRING ..... "myboard"
#define CONFIG_SOC_RS6130 .....

#ifndef CONFIG_HEAP_SIZE
#define CONFIG_HEAP_SIZE ..... (1024 * 40)
#endif

#ifndef CONFIG_KERNEL
#define CONFIG_KERNEL ..... 1
#endif

#endif /* CONFIG_BOARD_MYBOARD */
#endif /* _MYBOARD_CONFIG_H */
```

图 5-5 在 myboard_config.h 中修改相关的宏

6. 在工程的编译配置选项框中选择 Compiler 选项（编译选项配置请参考《RS6x_7x_SDK_参考手册》），在如下图 5-6 所示的 Compiler 和 Assembler 的全局宏定义选项框中将自己定义的模板的配置宏 CONFIG_BOARD_MYBOARD 设置为 1。

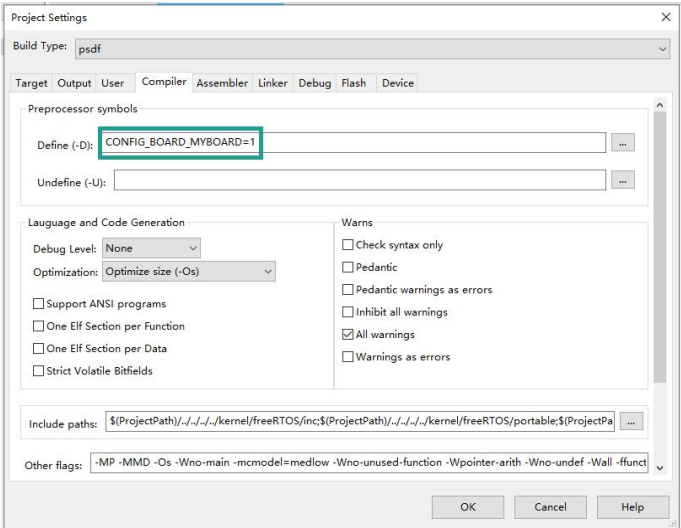


图 5-6 配置自定义模板全局定义宏

关于更多开发模板创建的方法，请参考《RS6x_7x_SDK_参考手册》。

5.3 通信功能移植

通信功能移植需要满足 SDK 设计准则，本章将介绍如何使用开发模板进行通信配置。其他外设和 CPU 的配置请参考《RS6x_7x_SDK_参考手册》和对应的 Demo。

5.3.1 通信接口功能配置

强烈建议开发者使用 SDK 中提供的 shell/LOG/HIF 三种通信协议，而不要使用底层驱动进行通信。这三种应用层协议可通过修改 5.2 中开发模板中的设置，映射至不同的硬件上。

表 5-1 shell/LOG/HIF 支持映射的硬件

	UART0 & UART1	SPI(Slave)	IIC
shell	√	×	×
LOG	√	×	×
HIF	√	√	√

SDK 对通信接口的使用具有如下限制：

- 1. shell/LOG 仅支持映射至 UART；
 - 2. HIF 与 shell/LOG 不能映射到同一个 UART 硬件上；
 - 3. shell 和 LOG 功能可以映射到同一个 UART 硬件上；
 - 4. HIF 仅能映射到一个硬件（UART0/UART1/SPI/IIC）上，不可在多个硬件上使用 HIF；
 - 5. 芯片仅支持通过 UART0、IIC、SPI 进行烧录功能；
 - 6. 若 shell/LOG/HIF 映射至了某个硬件，那么就不能使用底层接口对该硬件进行操作。例如 shell 映射至了 UART0，那么就不能使用串口驱动 API 对 UART0 进行操作，否则 shell 可能工作不正常。因此，若开发者希望使用底层驱动搭建自定义的协议，那么请不要将 shell/LOG/HIF 功能映射至这个硬件上。
- 综上所述，推荐开发者使用表 5-2 所列举的组合之一，进行通信功能配置，并且使用 HIF 协议进行数据传输，避免直接使用串口底层驱动。

表 5-2 推荐的通信功能配置

	UART0	UART1	SPI(Slave) or IIC
双串口配置 1	shell/LOG	HIF	DISABLED
双串口配置 2	HIF	shell/LOG	DISABLED
串口+SPI/IIC 配置 1	shell/LOG	DISABLED	HIF
无串口配置	DISABLED	DISABLED	HIF
单串口配置	HIF	DISABLED	DISABLED

注意 1：“无串口配置”仅支持通过 SPI/IIC 更新固件，其他配置均可通过 UART0 或 SPI 更新固件；

注意 2：可配置的 SPI 频率受 CPU 频率影响，具体可配置的频率请参考《RS6x_7x_SDK_参考手册》。请参考《RS6x_7x_SDK_参考手册》了解更多的 shell/LOG/HIF 信息和配置方法。

注意 3：MRS6130-P1806 仅支持双串口配置 2 和单串口配置。

若开发者需要在上位机接收 HIF 消息或了解更多关于 HIF 的移植，请参考《RS6x_7x_HIF_参考手册》，以获得 HIF 通信方式和示例代码。

关于 LOG 的详细移植请参考本文的 5.5.2 小节。

5.3.2 典型 HIF 应用场景

为保证数据的传输效率，建议开发者在进行数据通信时使用 HIF 通信协议。HIF 映射在不同的硬件时可支持不同的目标传输速率，开发者可根据应用需求选择。表 5-3 列出了典型应用下，HIF 在不同硬件映射下进行数据传输的使用时间。典型接口的应用场景以及数量大小如下：

- 1. 上报 2D-DataCube：距离 FFT 长度为 256，多普勒 FFT 长度为 32，MIMO 配置为 2T4R。每帧上报当前所有天线的 2D-DataCube 数据，需要上报的数据大小为 256K；
- 2. 上报 1D-DataCube：距离 FFT 长度为 256，MIMO 配置为 2T4R，雷达配置为 1D-FFT 处理流程，每帧上报所有天线的 1D-FFT 频谱，需要上报的数据大小为 8K；
- 3. PointCloud：一个有效的点云目标需要上报的数据大小为 12byte。

表 5-3 典型场景下不同接口传输能力

	DataCube 256x32, 2T4R 非压缩、256K 数据	DataCube 256x64, 2T4R 一倍压缩、512K 数据	DataCube-1D 256x1, 2T4R 8K 数据	PointCloud 512 个
SPI@60M bps (开发板、单线 SPI)	80ms	160ms	2.5ms	2ms
SPI@60M bps (QUAD-SPI)	30ms	40ms	0.625ms	0.5ms
UART@1M baud rate	3000ms	6000ms	95ms	50ms
UART@2M baud rate	1500ms	3000ms	47.5ms	25ms

这里需要注意的是，RS6x_7x 系列芯片支持 QUAD-SPI，若开发者需要更高的帧率支持，可选用支持 QUAD-SPI 的接口芯片，以实现更高速率的 DataCube 数据上报。若需要支持 QUAD-SPI，需要开发者在 SDK 的开发模板中将 SPI 配置为 QUAD-SPI。

若开发者希望通过 UART 连接 PC 进行数据上报，需要重点关注该 PC 的 USB 接口是否支持 2M 的串口波特率，以免出现数据无法上报的情况。

5.3.3 通信硬件 IO 配置

UART1 支持配置至 PA0/PA1（默认）和 PA8/PA9，SPI 仅支持一种 PinMux 配置。开发者需要根据硬件原理图，在板级配置文件中“myboard.c”中修改 PINMUX 管脚，如图 5-7 所示。更详细的配置方法请参考《RS6x_7x_SDK_参考手册》。

```
DEFINE_PINMUX_WEAK(uart1, 2,
{
    GPIO_PORT_A, GPIO_PIN_0, {GPIOA_P0_F3_UART1_TX, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    GPIO_PORT_A, GPIO_PIN_1, {GPIOA_P1_F3_UART1_RX, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
});

DEFINE_PINMUX_WEAK(uart1_deinit, 2,
{
    GPIO_PORT_A, GPIO_PIN_0, {GPIOx_Pn_F15_DIS, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    GPIO_PORT_A, GPIO_PIN_1, {GPIOx_Pn_F15_DIS, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
});
```

图 5-7 UART PinMux 修改

5.4 雷达功能移植

请参考《RS6x_7x_毫米波雷达_Sample_参考手册》，了解如何基于 radar framework，快速进行毫米波雷达的软件功能移植。

5.5 调试接口

SDK 提供了三种调试方法，可满足开发者对时序、日志和复杂数据传输的调试需求。

5.5.1 TRACE_IO

TRACE_IO 提供了一组 API，通过为 IO 设置不同的脉冲波形，用于对代码的执行速度进行测量和时序的调试。使用时需准备逻辑分析仪或者示波器，来观测引脚的电压状态。使用时，需定义宏

“CONFIG_LOG_IO_TAG_MASK”和“CONFIG_LOG_IO_MASK”，使用规则如下：

1. 宏 CONFIG_LOG_IO_TAG_MASK 的作用是在使用 TRACE_IO 时的标志，定义值仅需满足 uint32_t 可表示范围，例如定义为 0x1000 即可；
2. CONFIG_LOG_IO_MASK 定义与使用的引脚相关，从 bit0~bit11 每一位对应是否使用 PA0~PA11，使用某一引脚作为 TRACE_IO 时，需要将对应的 bit 置 1，例如使用 PA0 作为 TRACE_IO 时，需定义为 0B0000000000001 (0x01)。

选择 TRACE_IO 时，需要注意：

- 仅 PA 组可作为 TRACE_IO 选择的引脚，PB0 组驱动暂不支持；
- 在 TRACE_IO 的引脚选择时，需要查看该引脚是否已被其他功能使用，例如使用 UART-SPI/UART-I2C 模式时，PA6 被 IRQ 使用，如果此时仍使用 PA6 作为 TRACE_IO 引脚，可能引起功能异常；
- 开发者如果使用 MRS6130-P1806 模组，仅可使用 PA0 作为 TRACE_IO，但 PA0 默认被 UART1 使用，因此使用 TRACE_IO 时，不可使用 UART1。

使用 TRACE_IO 时，不需要应用层进行初始化等操作，但需要在代码中包含头文件“log.h”，使用接口 LOG_IO(uint32_t tag, uint8_t num, uint8_t val, uint8_t flip)来控制 IO 实现各类操作，各参数定义如下：

1. Tag：TRACE_IO 的条件判断，接口内部会将参数与前面定义的宏“CONFIG_LOG_IO_TAG_MASK”进行比较，两值相同时才会执行 IO 操作；
2. Num：选择控制的 IO 口，例如输入参数 0 时，会对 PA0 执行相应的操作；
3. Val：输入 0 或 1 分别对应，对应的控制 Num 选择的 IO 进行拉低或者拉高电平的操作；
4. Flip：当此参数不为 0 时，控制 Num 选择的 IO 进行此参数数量的脉冲，需要注意：
 - 当此参数为 0 时，根据 Val 的值控制 IO 进行电平的拉高或者拉低；
 - 当此参数不为 0 时，输入参数 Val 的值无效，IO 将在完成 Flip 数量后的脉冲后变为低电平。

在选择引脚进行 TRACE_IO 测试前，请参考《RS6xxx_硬件设计和 PCB Layout_指导手册》。

5.5.2 日志系统

SDK 提供了一组日志系统的 API，相比于 TRACE_IO，日志系统可以更具体的得到调试信息。使用日志系统前，需要在工程中的 prj_config.h 中做以下预备操作：

1. 定义宏“CONFIG_PRINTF”，此宏为日志系统总开关，当使用日志时，需定义为 1；
2. 定义宏“CONFIG_PRINTF_UART_BAUDRATE”，此宏控制使用日志时的波特率，若开发者未定义，默认定义为“115200”，开发者可根据实际进行修改；
3. 定义宏“CONFIG_PRINTF_UART_NUM”，此宏控制使用日志时使用的 UART 口，使用“UART-SPI/IIC”通信模式时，仅可定义为 0 (UART0) 作为日志 0，使用“DUAL-UART”通信模式时，建议定义为 1 (UART1)；

需要注意的是：

- 日志所使用的 UART 口和 HIF 使用的 UART 口不能为同一个；
4. 在#include“log.h”之前定义宏“LOG_LEVEL”，此宏控制日志系统的输出等级，通过日志系统调用接口，以及设定的日志等级，可实现调试信息的输出控制，配置高的日志等级可输出低等级的接口输出，各等级的配置值与定义如下：
 - 配置为 0，不会输出任何信息；
 - 配置为 1，输出代码中功能受阻或数据异常的错误信息；
 - 配置为 2，输出代码中不影响当前功能但需注意的警告信息；
 - 配置为 3，输出代码中常规的运行信息，例如状态查看等；
 - 配置为 4，输出代码中的调试信息，推荐在开发阶段使用；
 5. 在#include“log.h”之前定义宏“LOG_MODULE”，此宏需定义为字符串的形式，在调用日志接口时，会输出此宏信息，建议开发者根据文件名定义此宏，可帮助开发者从日志信息快速定位代码文件；
 6. 包含头文件“log.h”。

参考工程中，步骤 1~3 均在

[“project/mmwave/common/radar_framework/inc/radar_framework_default_config.h”](#) 中定义。

使用日志系统输出信息时，开发者应根据信息类型选用接口，搭配上述步骤 4 中定义的日志等级使用，各日志接口的使用如下：

1. LOG_PRINT：使用方法和 C 语言库中的标准函数“printf”一致，当 LOG_LEVEL 配置大于 0 时，调用此接口可输出日志信息；
2. LOG_ERR：使用方法和 LOG_PRINT 一致，当 LOG_LEVEL 配置大于等于 LEVEL_ERR 时，调用此接口可输出日志信息，与 LOG_PRINT 的区别在于，调用此接口会在日志信息前自动添加“[ERR “LOG_MODULE”]”；
3. LOG_WRN：和 LOG_ERR 类似，LOG_LEVEL 配置大于等于 LEVEL_WRN 时，此接口可输出日志信息，输出时会在日志信息前自动添加 “[WRN “LOG_MODULE”]”；
4. LOG_INF：和 LOG_ERR 类似，LOG_LEVEL 配置大于等于 LEVEL_INF 时，此接口可输出日志信息，输出时会在日志信息前自动添加“[INF “LOG_MODULE”]”；

5. LOG_DBG：和 LOG_ERR 类似，LOG_LEVEL 配置大于等于 LEVEL_DBG 时，此接口可输出日志信息，输出时会在日志信息前自动添加"[DBG "LOG_MODULE"] "；

需要注意：

- 上述自动添加信息中的"LOG_MODULE"为宏“LOG_MODULE”定义的字符串，若未定义则不存在；
- 配置低的日志等级且调用日志等级高的接口时，表现仅为不输出日志信息，不会出现编译错误。

按上述完成日志系统的使用后，运行固件并使用串口调试工具查看 LOG 信息，选择上述步骤 2 和 3 定义的波特率和端口即可观测调试信息。

5.5.3 PSIC-调试协议

SDK 在 HIF 的 Payload 字段内定义了称为“PSIC-调试协议”的子协议，具有以下关键特性：

1. 属于 HIF 协议框架的子协议，具有固定的 HIF MsgID 为 0xC6，数据填充于 HIF 协议的 Payload 字段，开发者免于开发新的应用层协议；
2. 支持 byte/short/word/float/double 等多种类型数据的上报，支持数组上报；
3. 支持上报定点位宽，并在上位机还原定点位宽；
4. 支持定义不同的数据组织方法；
5. 支持灵活的数据标签，以字符串类型区分每种数据类型；
6. 至多支持 4090Bytes 数据上报；
7. 支持字符串上报，并在 RadarDebugTool 中显示字符串。

PSIC-调试协议与 HIF 协议 Payload 字段关系如下图所示：

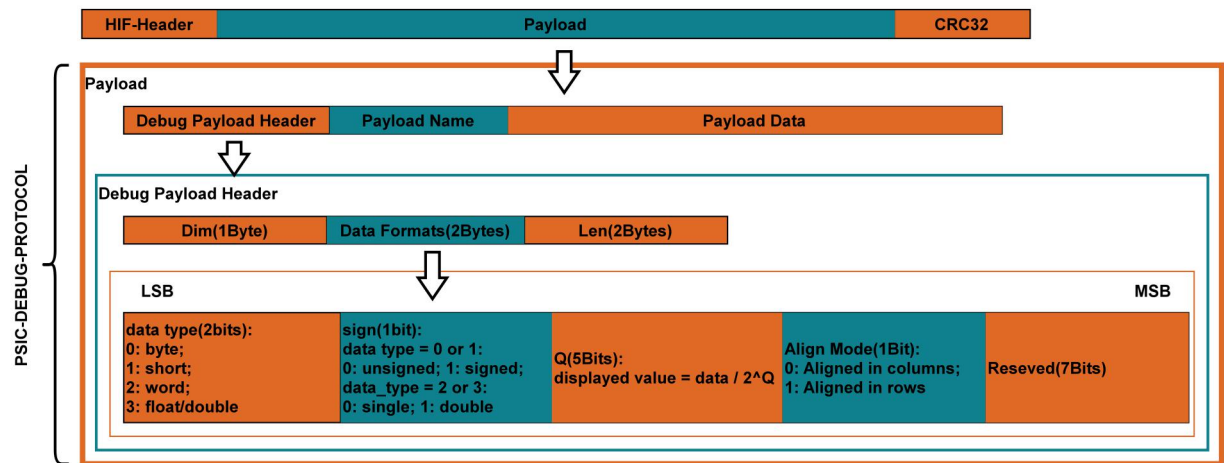


图 5-8 PSIC-调试协议

在参考工程中，调用接口函数 *radar_framework_report_debug_data()* 可以将数据按照 PSIC-调试协议的数据格式上报给上位机。本章节主要介绍各个 PSIC-调试协议中调试参数的意义。

PSIC-调试协议中的数据类型字段（Data Format）控制了数据的组织方式。其各标志位意义如下：

- Dim
 - 指示需要上报的数据的维度，例如笛卡尔坐标 x/y/z 为一个 3 维数据。当 Dim = 1/2/3 时，RadarDebugTool 支持对数据的可视化。
- Len

- 指示需要上报的多维数据的个数。Dim * Len 为上报数据的总个数。
- Align Mode（数据对齐方式）
 - 当 Align Mode=0 时，Payload Data 中数据优先填充 Len 个相同维度的数据，当某个维度数据填满后，再填充下一个维度的 Len 个数据，直到 Dim 维数据全部填充完成。

以填充 N 个 x/y/z 笛卡尔坐标为例，设置 Dim=3，Align Mode=0PSIC-调试协议中的数据排列如下所示：

x1	x2	...	xN	y1	y2	...	yN	z1	z2	...	zN
----	----	-----	----	----	----	-----	----	----	----	-----	----

- 当 Align Mode = 1 时，Payload Data 中数据优先填充 Dim 个不同维度的数据，再填充下一组 Dim 个不同维度的数据，直到 Len 个多维数据全部填充完成。

以填充 N 个 x/y/z 笛卡尔坐标为例，设置 Dim=3，Align Mode = 1，PSIC-调试协议中的数据排列如下所示：

x1	y1	z1	x2	y2	z2	...	xN	yN	zN
----	----	----	----	----	----	-----	----	----	----

- Q（定点化位宽）
 - RadarDebugTool 在解析 Payload Data 数据后，会将数据除以 2^Q 进行显示和存储。这在上报的数据是定点化的数据时很有用，当上报数据为浮点类型时，一般需要将 Q 设置为 0。
- Data Type（数据类型）
 - 指示每一个元素的数据类型，支持的数据类型包括 byte/short/word/float(double)类型；
- Sign（符号标志、浮点类型）
 - 当 Data Type 为 byte/short/word 时，Sign=1，表示 Data Payload 中所有为无符号数，Sign=0，表示 Data Payload 中所有数据均在上位机被视作无符号数；
 - 当 Data Type 为 float(double)类型时，Sign = 1，表示上报类型为 double，Sign = 0，表示上报类型为 float。
- Payload Name
 - Payload Name 指示了该调试数据的标签，这是一个字符串，必须以'\0'作为结尾。此标签决定该段数据在 RadarDebugTool 上显示和存储的名称；
- Payload Data
 - 根据 Align Mode/Dim/Len 填充数据。

以“Tracking”工程中的 frame_info 上报为例，其在 RadarDebugTool 上显示如图 5-9 所示，右侧的 LOG 工作区可显示接收到的原始十六进制数据。以某一帧工具收到的码流为例：

A5 2C 32 C6 26 10 01 01 00 0B 00 66 72 61 6D 65 5F 69 6E 66 6F 00 60 00 04 00 F4 01 00 19 00 01 40 06 20 00 00 00 07 00 00 00 00 00 76 03 54 FA

码流拆解：

- C6：MSG_ID，位于图 5-8 中的“HIF-Header”段，使用 PSIC-调试协议上报的数据均为 C6；
- 01 01 00 0B 00：对应图 5-8 中的“Debug Payload Header”；

- **0x01** 表示 Dim 的数据维度为 1；
- **0x01** 分别表示：
 - Data Type 为 01（2bits），为 short 型；
 - Sign 为 0（1bit），为无符号数；
 - Q 为 00000（5bits），为 0；
- **0x00** 表示 Align Mode 的数据对齐方式为 0；
- **0x0B** 和 **0x00** 组合表示数据长度 Len，低十六位在前，因此组合为 0x000B，表示当前数据长度为 11。

66 72 61 6D 65 5F 69 6E 66 6F 00: 对应图 5-8 中的“Payload Name”，通过 ASCII 对应为字符串“frame_info”；

60 00 04 00 F4 01 00 19 00 01 40 06 20 00 00 00 07 00 00 00 00 00: 对应图 5-8 中的“Payload Data”，采用小端模式，数据长度等信息都对应上述“Debug Payload Header”。

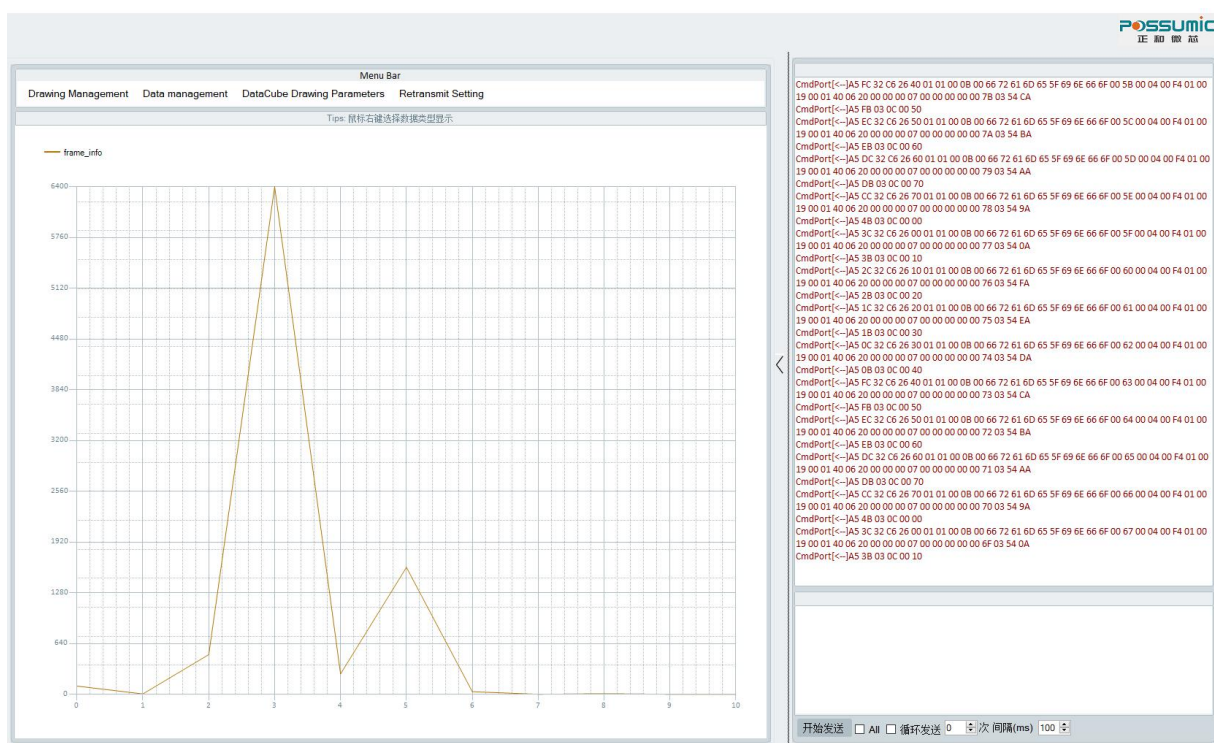


图 5-9 frame_info 工具显示

5.6 可靠性设计

mmwave 示例工程中使用 4 种调试手段以确保雷达正在正常工作，其中 2 种调试手段需要在 HOST 端进行检测。

5.6.1 看门狗

使用宏 CONFIG_MMW_WATCHDOG 使能系统看门狗，在启用看门狗的基础上，使用 CONFIG_WDG_TIMEOUT_MS 宏确定看门狗的复位时间，默认为 5s。

使用三个接口以控制看门狗：

- watchdog_init(): 创建看门狗设备、初始化看门狗并启动看门狗；

- watchdog_stop(): 停止看门狗功能;
- watchdog_open(): 在调用 watchdog_stop()后启动看门狗;
- mmw_watchdog_feed(): 喂狗。

5.6.2 Task Monitor

使用 Task Monitor 监控 mmw_ctrl 中的雷达发帧控制是否在正常进行。

- mmw_loop_task_time_init(): 创建一个 Task Monitor, 并启动监控;
- mmw_loop_task_monitor_update(): 根据帧周期优化 Task Monitor 的监控时间;
- mmw_loop_task_time_reset(): 在每个回调中调用此函数以重置 Task Monitor, 在 Task Monitor 停止后调用此函数以重新启动 Task Monitor;
- mmw_loop_task_time_stop(): 停止 Task Monitor 功能。

5.6.3 心跳包

SDK 支持通过 MsgId=0xFE 的 HIF 报文定时上报心跳包。心跳包机制用于应用不需要周期性上报目标时, 上位机可通过心跳包判断雷达是否仍然存活。

一个典型的心跳包码流的 HIF 部分具备 TLV 格式:

```
A5 D6 32 FE 14 40 00 04 00 00 00 00 01 04 8B 0E 00 00 02 02 00 00 03 02 00 00 ED EB E9 BB
```

上述码流意义为:

32 FE 14 40 为 HIF 包头, 表示 MsgId=0xFE, 长度为 0x014 字节;

00 04 00 00 00 00 是一组 TLV 数据:

Tag 为 1 字节: 00 表示 RESPONSE_CNT;

Length 为 1 字节: 04 表示后续的 Value 长度为 4 字节;

Value 为 0;

心跳包中 Tag 定义如下:

RESPONSE_CNT = 0: Value 值为 HIF 回复 Response 报文的次数

FRAME_INDEX=1: Value 值为帧序号 (从 1 开始)

MOTION_PT_NUM=2: Value 值为本帧移动点云计数;

PRESENT_PT_NUM=3: Value 值为本帧存在点云的计数值。

关于 HIF 消息格式的定义, 请参考《RS6x_7x_HIF 开发_快速入门》。

5.6.4 Response Count

HOST 每发一个命令, Device 进行一次 Response, 随后 Response Count 自增。HOST 可定期通过 HIF 命令获取 Response Count, 和上一次进行对比。如果变小或获取失败、通信异常, 则认为雷达异常, 可通过 HOST 控制雷达重启。

获取 Response Count 的 HIF 消息的 MsgId=0x01, SubId=0x41。雷达回复的 HIF 消息 Payload 为 1Byte 的 Status + 4Bytes 的 Response Count。

参考码流：

Host --> 雷达：

A5 43 15 01 01 00 41 A9 FE FE FF

雷达 --> HOST

A5 42 12 01 05 00 00 01 00 00 00 ED FD FA FF

12 01 05 00: HIF Header

00 01 00 00 00: status=0, 表示接收成功, Response Count=0x00000001 表示收到此 CMD 之前已经进行了一次 Response 回复。

5.7 嵌入式资源配置与验证

5.7.1 代码资源配置方式

当开发者进行用户代码开发时，默认代码均运行在 XIP 中，下面介绍两种方式将代码修改至在 SRAM 中运行：

1. 在链接文件（link file）中的 isram_text 和 isram_rodata 中加入对应文件的通配符：

```
isram_text : {
    . = ALIGN(0x4);
    __s_isram_text = .;

    /* add_new_file 前后的*为文件通配符，建议前面一定要加，后面的*可以不加 */
    *add_new_file*.o(*.text*)

    . = ALIGN(0x4);
    __e_isram_text = .;
}> ISRAM
```

2. 在函数定义前添加 __sram_text 或 __keep_sram_text 修饰符将此函数链接至 SRAM，如：__sram_text void my_func();

链接文件位于工程配置指定的路径下：

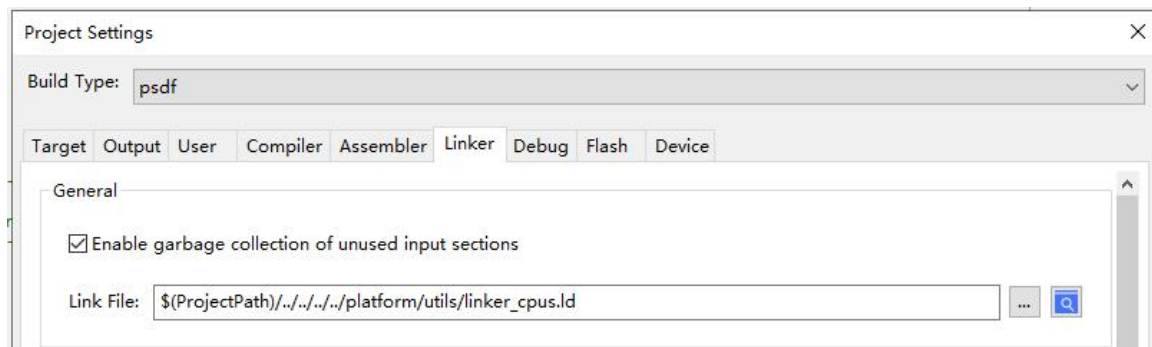


图 5-10 链接文档指定路径

在代码编译时，使用“__sram_text”修饰的函数，无论定义的函数或变量是否被使用，都会被编译，因此可能造成资源的浪费，为避免这种情况，为开发者提供两种解决方法：

1. 对于具体的功能，使用宏来控制功能是否开启，并用此宏控制函数或变量的定义与使用，避免未使用此功能时相关函数和变量占用资源；
2. 未被其他模块调用，且需要使用“__sram_text”定义在 SRAM 中时，需使用说明符“static”进行修饰，即可实现未使用函数时，不将此函数进行编译，避免资源浪费。

Section Headers:

[Nr]	Name	Type	Addr	Off	Size	ES	Flg	Lk	Inf	AI
[0]		NULL	00000000	000000	000000	00		0	0	0
[1]	head_pad	PROGBITS	09000000	011000	000020	00	WA	0	0	1
[2]	isram_text	PROGBITS	00500000	002000	00da2c	00	WAX	0	0	64
[3]	isram_rodatab	PROGBITS	0050da30	00fa30	000c48	00	A	0	0	8
[4]	ipsram_text	PROGBITS	0d000000	023654	000000	00	W	0	0	1
[5]	ixip_text	PROGBITS	09000020	011020	00fb94	00	AX	0	0	2
[6]	ixip_rodatab	PROGBITS	0900fbb8	020bb8	0023f0	00	A	0	0	8
[7]	dsram_data	PROGBITS	10500000	023000	000654	00	WA	0	0	8
[8]	dsram_bss	NOBITS	10500658	023654	000a98	00	WA	0	0	8
[9]	dsram_stack	NOBITS	105010f0	023654	00c800	00	WA	0	0	16
[10]	ipsram_rodatab	PROGBITS	0d000000	023654	000000	00	W	0	0	1
[11]	dpsram_data	PROGBITS	1d100000	023654	000000	00	W	0	0	1
[12]	dpsram_bss	PROGBITS	1d100000	023654	000000	00	W	0	0	1
[13]	.info	PROGBITS	00000000	001000	0000ce	00	A	0	0	4
[14]	.comment	PROGBITS	00000000	023654	000093	01	MS	0	0	1
[15]	.riscv.attributes	RISCV_ATTRIBUTE	00000000	0236e7	000125	00		0	0	1
[16]	.debug_info	PROGBITS	00000000	02380c	0040e4	00		0	0	1
[17]	.debug_abbrev	PROGBITS	00000000	0278f0	0018c6	00		0	0	1
[18]	.debug_loclists	PROGBITS	00000000	0291b6	004dd6	00		0	0	1
[19]	.debug_aranges	PROGBITS	00000000	02df8c	000278	00		0	0	1
[20]	.debug_line	PROGBITS	00000000	02e204	009295	00		0	0	1
[21]	.debug_str	PROGBITS	00000000	037499	0008c1	01	MS	0	0	1
[22]	.debug_line_str	PROGBITS	00000000	037d5a	000406	01	MS	0	0	1
[23]	.debug_frame	PROGBITS	00000000	038160	00034c	00		0	0	4
[24]	.debug_rnglists	PROGBITS	00000000	0384ac	0007b6	00		0	0	1
[25]	.symtab	SYMTAB	00000000	038c64	0091e0	10		26	1236	4
[26]	.strtab	STRTAB	00000000	041e44	0070f6	00		0	0	1
[27]	.shstrtab	STRTAB	00000000	048f3a	000142	00		0	0	1

图 5-11 map 文件内存分布

更多的关于代码运行位置的配置方法请参考《RS6x_7x_SDK_参考手册》。

5.7.2 嵌入式资源验证

上一小节介绍了开发者在开发过程中对如何配置代码资源，以及详细查找某一函数的资源使用情况，本节主要介绍如何进行整体的资源统计。

在图 4-4 的 img 打包信息中统计了使用的 SRAM 和最大可使用的 SRAM，另外 SDK 为开发者提供了更为详细的资源统计功能，开启时需要在“prj_config”中定义宏“CONFIG_DEBUG_SIZE_LEVEL”，可配置两个等级，效果如下：

配置“CONFIG_DEBUG_SIZE_LEVEL”为 1 的结果如图 5-12 所示，在编译完成后，会分别输出引导程序、主应用程序和芯片底层配置所用的 SRAM 大小。

- (4) 在此路径下打开 CMD 或其他命令行工具，输入命令：“python find_xip_refs.py --asm psdf.asm radar_framework_2d_frame_cb”，其中“radar_framework_2d_frame_cb”为待测试的函数，开发者可根据实际使用修改；
 - (5) 等待脚本运行完后，会显示执行在 XIP 中的函数，并将结果保存在与接口名相同的.csv 文件中，开发者也可通过.csv 文件查询具体信息。如果查询的函数未在“psdf.asm”中包含，则会提示“未在文件中找到函数”。
2. 直接查看编译后工程路径下 Lst 文件夹子目录下的“psdf.map”文件，确认图 5-11 中 SRAM 的内存地址，并在文件中搜索需要查询的函数，判断此函数地址是否在 SRAM 中。例如：在示例文件中，函数“radar_framework_2d_frame_cb”的地址为“0x0050d866”，而 SRAM 的地址范围为“0x00500000~0x0050da2c”，因此可判断当前函数在 SRAM 中。

5.7.3 计算时间测量

本小节为开发者提供代码计算时间的测量方式，通过 SDK 目前可采用两种方式：

1. 使用 TRACE_IO 观测计算时间，根据 5.5.1 中完成相关配置，在待测量代码段前拉高选用的 IO，在结束计算后拉低选用的 IO，通过逻辑分析仪或者示波器观测选用 IO 的高电平持续时间，此时间即为这段代码的计算时间，以 PA0 为示例：

```
#include "log.h"
#define CONFIG_LOG_IO_TAG_MASK      (0x1000)
#define CONFIG_LOG_IO_MASK          (0x01) /* 使能 PA0 作为 LOG */

LOG_IO(CONFIG_LOG_IO_TAG_MASK, 0, 1, 0); /* 拉高 PA0 */
/* 执行待测试的代码 */
LOG_IO(CONFIG_LOG_IO_TAG_MASK, 0, 0, 0); /* 拉低 PA0 */
```

2. 通过 SDK 提供的 HAL 库接口测量计算时间，在包含了头文件“hal_board.h”后，可通过接口 HAL_BOARD_GetTime(HAL_TimeType_t timeType)获取当前的时间节点。根据输入参数 timeType 可控制返回时间节点的单位，目前支持：
 - HAL_TIME_CYCLE：以 CPU 周期为单位返回当前时间节点；
 - HAL_TIME_US：以微秒为单位返回当前时间节点；
 - HAL_TIME_MS：以毫秒为单位返回当前时间节点；
 - HAL_TIME_S：以秒为单位返回当前时间节点；

通过使用待测代码段前后的时间节点差，作为计算时间的测量值，通过日志系统或者调试协议可将计算时间可视化，以日志系统为例：

```
#define LOG_LEVEL 1
#define LOG_MODULE "my_module"
#include "log.h"
#include "hal_board.h"
uint32_t time_start = HAL_BOARD_GetTime(HAL_TIME_US);
/* 执行待测试的代码 */
uint32_t time_end = HAL_BOARD_GetTime(HAL_TIME_US);
LOG_PRINT("Calculate the required time:%d(us)\n", time_start - time_end);
```

5.8 低功耗功能移植与验证

5.8.1 低功耗模式配置

通过配置宏“CONFIG_PM”为 1 打开休眠唤醒模式。在休眠唤醒模式中，雷达在完成计算任务后，通过系统的 idle 线程控制设备进入低功耗状态。进入低功耗状态后，雷达在完成 Frame 发送后进入休眠状态。相关配置请参考图 5-14，关于休眠唤醒的具体功能请参考《RS6x_7x_SDK_参考手册》。

```
/* power mange config
 */
/**
 * @brief If enable 'CONFIG_PM', after CPU finished processm, CPU will shut down and consumes a current of 10uA between frames.
 */
#ifndef CONFIG_PM
#define CONFIG_PM 0
#endif
#define CONFIG_PMU_EXTLDO_SLEEP_SW_SUBMODE 1
```

图 5-14 低功耗和 CPU 性能配置

当使用低功耗模式时，需配置并使用 wkio。当系统休眠时，外设处于掉电状态，普通 IO 无法保持输出或接收主控的唤醒信号。但 wkio 可在系统休眠时仍然保持指定的电平输出和输入中断。wkio 的示例位于：[psdf_sdk/project/driver/wkio](https://www.possuimic.com/psdf_sdk/project/driver/wkio)。

5.8.2 功耗验证

下面介绍开发者如何对功耗进行测试，进行功耗测试前请准备电流表，功耗测试流程如下：

1. 将图 2-1 中右下角的 VDD3V3-VCC 断开，并连接 VDD13-V13，若使用 MRS6130-AIP-DEV-V1 或 MRSx241-DEV-V1 开发板，仅断开 VDD3V3-VCC 即可；
2. 将硬件面对空旷的地方，其目的在于减少点云计算时间，使硬件更快的进入休眠；
3. 将电源正极接入底板 VCC，负极接入底板 GND。正负极接反并启动电源时可能导致硬件损坏，在启动电源前请进行检查；
4. 启动电源，并使用电源配套工具观测电流，以示例工程为例，一帧的电流中应该观测到发波-CPU 计算-HIF 传数-休眠（开启 PM）的波形，建议开发者统计：
 - 平均功耗：建议累计 2 分钟以上的电流波形，并记录平均电流或功率，进行平均功耗的测试。
 - 休眠电流：开启 PM 后，对休眠状态下的平均电流进行记录，理论上休眠底电流不会超过 10uA。

需要注意：

- 1. 满足以下情况时：（1）开启 PM，（2）使用 SPI 通信，（3）存在 HIF 传数操作；进行功耗测试时需要连接上位机工具并开启扫描，否则 HIF 会影响进入休眠状态时间。
- 2. 由于电源的性能不一，在进行功能测试时可能存在以下现象：进入休眠后，底电流持续升高，导致休眠时间越长，休眠底电流越高。若出现这种现象，为获得更精确的休眠电流，建议增大帧周期，使休眠时间在 500ms 以上时再进行测试；
- 3. 进行功耗测试时，需将图 4-1 中 POWER 旁的 VCC-5V 的单刀双掷开关拨到 VDD3V3，否则测试的电流不准确。

5.9 SDK 其他修改

除上述通信接口的修改以外，在移植至目标硬件可能还存在更多细节的修改。针对这些功能，SDK 中提供了对应的示例程序，开发者可参阅这些示例程序，快速完成系统功能的配置。

- **OTA**：开发者可设计两种 OTA 方式：BURN 升级和 OTA 升级。BURN 升级使用烧录的方式，直接对雷达 Flash 区进行操作，而 OTA 升级将固件烧录在 Flash 的备份区，重启系统后可进行新旧 img 交换。两者的重要差异在于：使用 BURN 升级的方式，仅需要 1 倍最大 img 大小的 Flash，而使用 OTA 升级的方式则需要至少 2 倍 img 大小的 Flash。更多 OTA 的信息请参考《RS6x_7x_固件下载_开发手册》。
- **CPU 性能配置**：通过在“prj.config.h”中新增“CONFIG_SOC_CLOCK_CFG”宏定义，可修改预设的 CPU 时钟，请参考图 5-14。预设的时钟频率如表 5-4 所示。

表 5-4 默认的 CPU 频率

	CONFIG_SOC_CLOCK_CFG 取值	CPU 运行频率
RS6130-P1806	未指定	128MHz
RS6130-P1812	未指定	128MHz
RS6240	未指定	256MHz
RS6241-P2828	未指定	256MHz
RS6241-P2840	未指定	256MHz

可选择的 CPU 时钟请参考文件'[psdf_sdk/platform/soc/rs613x/rs6130_config.h](#)'和'[psdf_sdk/platform/soc/rs624x/rs6240_config.h](#)'以及《RS6x_7x_SDK_参考手册》。

6 雷达调试方法

本章介绍如何在图 1-1 所示的开发流程中有效使用 RadarDebugTool 工具，解决雷达系统调试过程数据吞吐量大、数据可视化能力差的问题，帮助开发者快速定位和分析问题。在设计与评估阶段，RadarAnalysisTool 可帮助开发者验证波形配置和进行算法设计，具体评估方法请参考《RS6x_7x_毫米波雷达评估_快速入门》。

在开发阶段，RadarDebugTool 工具提供了如下功能：

1. 高效的 DataCube 获取与可视化能力：在单线制 SPI 接口的硬件条件下，最高支持以 10Hz 帧率上报 256K DataCube，并为 DataCube 提供多种可视化手段以查找问题；
2. 多种调试信息可视化手段：支持自定义调试信息上报、可视化和存储功能；

若需要了解更多的 RadarDebugTool 的使用细节，请参考《RS6x_7x_RadarDebugTool_使用手册》。

6.1 雷达配置接口的生成

开发者在设计与评估阶段，通过 RadarAnalysisTool 确认了波形配置和算法原型，可一键生成硬件配置函数调用示例，并导入到工程中。

6.2 测试与验证

开发者在测试与验证过程中可利用 5.5.3 提供的 PSIC-调试协议进行任意数据的可视化功能，以图表的形式实时监控自定义的软件运行状态，利用 RadarDebugTool 的实时监控和存储 DataCube 的功能，及时发现测试过程中的异常数据，帮助开发者定位系统 BUG。

RadarDebugTool 的数据可视化功能可参考《RS6x_7x_RadarDebugTool_使用手册》和《RS6x_7x_毫米波雷达_Sample_参考手册》。

7 Q&A

Q：项目前期基于开发板开发验证时，应选择 UART-SPI 模式还是 DUAL-UART 模式？

A：由于雷达调试数据具有数据量大的特点，建议基于开发板开发时，使用 **UART-SPI 模式**，可节约数据传输时间，支持更高帧率的雷达发波周期。

Q：雷达的 LOG 串口为什么打印“mmw_process_fft_ant_f32 call failed, err_code: 4”？

A：此错误是由于雷达波形发送完成到下一帧的 Frame 之间时间过短，硬件在发波时将数据写入 DataCube 时软件仍然在访问 DataCube 从而发生冲突，这可能导致当前数据处理流程数据异常。可通过限制最大点云数、降低帧率等方法来进行规避此问题。

Q：编译过程中提示找不到 xxx.o，导致编译不过

A：当代码目录结构发生了变化时，残余的链接信息不能清除干净，这时需要开发者点击‘Clean Project’按钮，或手动删除工程目录下的‘Lst’和‘Obj’目录，重新编译工程。

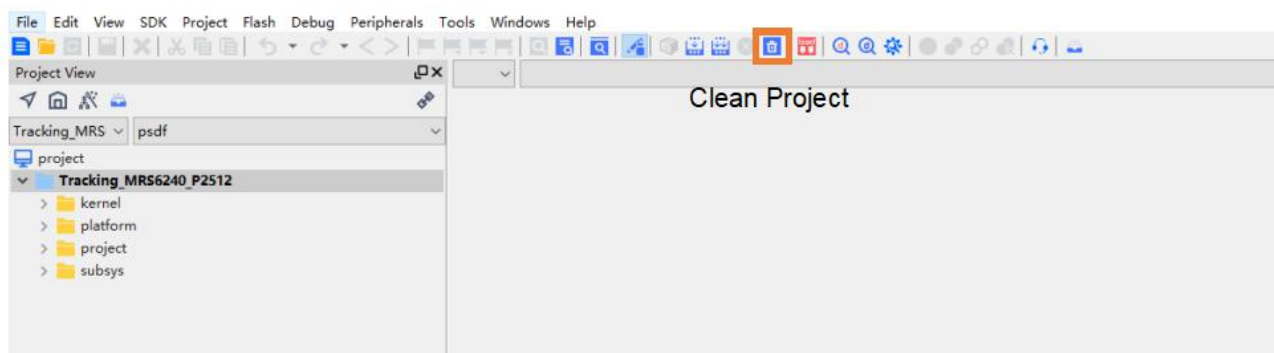


图 7-1 Clean Project

Q：如何连接雷达与其它嵌入式硬件进行开发

A：可以通过开发板两边的排针孔与其它嵌入式硬件连接，UART0 对应 PB0-TX、PB1-RX，UART1 对应 PA0-TX、PA1-RX，SPI 对应 PA0-CS、PA1-SCLK、PA2-MOSI、PA3-MISO，I2C 对应 PA4-SDA、PA5-SCL。

在连接之前，需要确认连接模组引脚与开发板硬件的电阻是否焊接完好，下图绿色框选处为当前说明的电阻，这些电阻以 2 个为一组进行编号。

对于 MRS6240-AIP-DEV-V1，共 6 组电阻，从左往右依次为 0A、0B、1A、1B、...、4A、4B、5B、5A，0 号对应 PA5、1 号对应 PA4、...、5 号对应 PA0，A 对应开发板上的排针孔，B 对应 FUNC SEL 或转接芯片的引脚，其中 0B 和 2B 直接连接转接芯片。

对于 MRS6130-AIP-DEV-V1，共 7 组电阻，从上至下依次为 0A、0B、1A、1B、...、6A、6B，0 号对应 PA5、1 号对应 PA4、...、5 号对应 PA0，6 号比较特殊，在后面对其说明。A 对应开发板上的排针孔，B 对应 FUNC SEL 或转接芯片的引脚，其中 0B 和 2B 直接连接转接芯片。6A 和 6B 连接 MRS6130-P1806 模组的 POUT 引脚（2 号引脚），6A 将 POUT 连接至开发板 PA0，6B 将 POUT 连接至开发板 PA6。

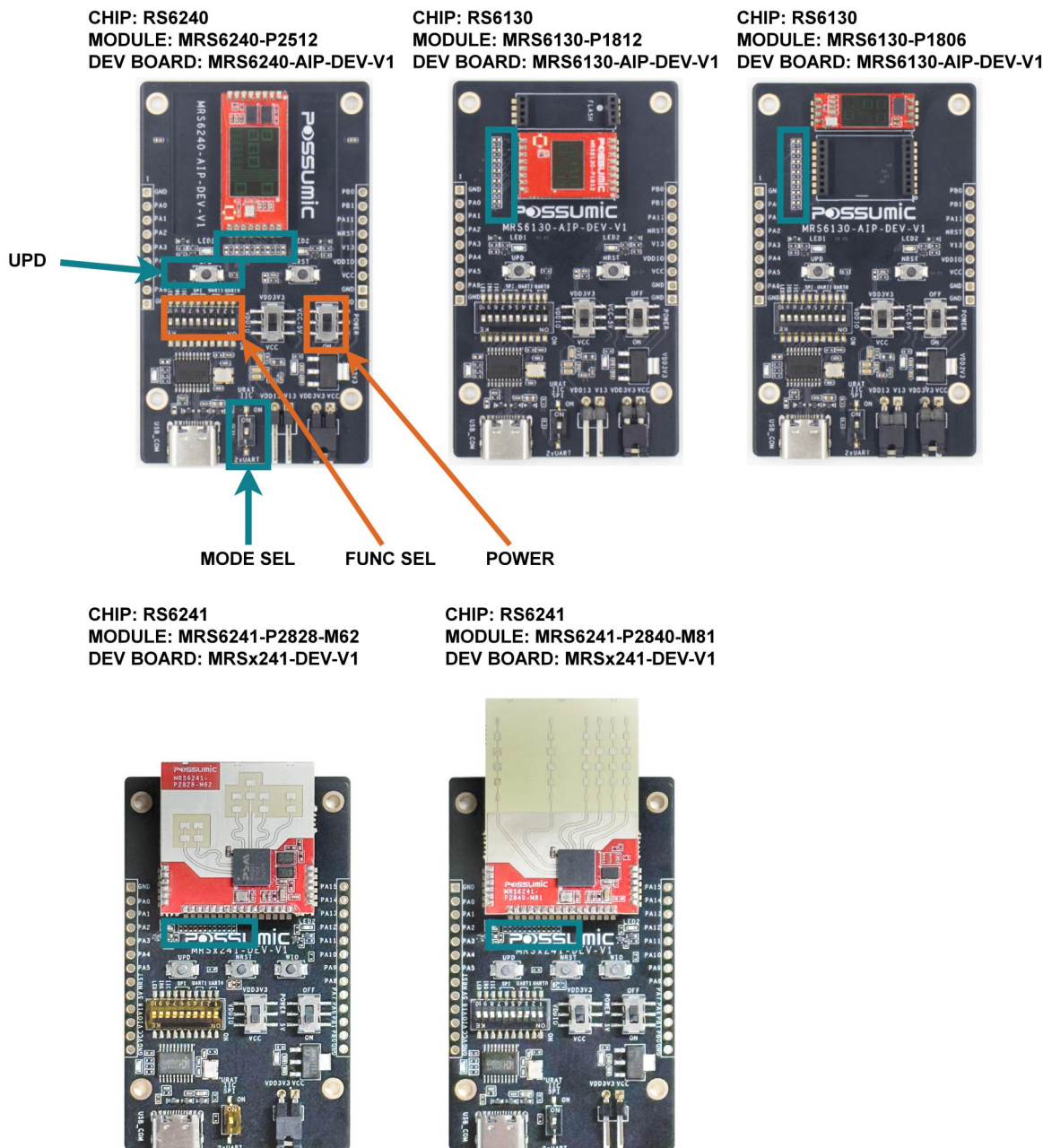


图 7-2 连接模组引脚与开发板硬件的电阻

如果需要开发雷达的 Host，可以参考《RX6x_7x_HIF_参考手册》。

免责声明和著作权声明

本司在本文档中提供的信息力求准确，但并不保证内容完全无误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或不遵守本文档使用说明而进行的任何不当使用行为（包括但不限于如超压，超频，超温使用）所造成的损失，本司概不负责。

本文中的内容，包括但不限于供参考的 URL 地址，由于产品版本升级或其他原因，与实际内容存在差异，如有变更，恕不另行通知。文档“按现状”提供，不负任何担保责任，包括对适销性、适用于特定用途或非侵权性的任何担保，和任何提案、规格或样品在他处提到的任何担保。

本文档不负任何责任，包括使用本文档内信息产生的侵犯任何专利权行为的责任。本文档在此未以禁止反言或其他方式授予任何知识产权使用许可，不管是明示许可还是暗示许可。

版权所有©2026 珠海正和微芯科技有限公司。保留所有权利。